LLM Interpretability Part 1

ELL8299 · COL8383

Tanmoy Chakraborty
Professor, IIT Delhi
https://tanmoychak.com/

LCS LABORATORY FOR COMPUTATIONAL SOCIAL SYSTEMS

भारतीय प्रौद्योगिकी संस्थान दिल्ली INDIAN INSTITUTE OF TECHNOLOGY DELHI

LLM Interpretability – Objectives and Motivations

- We characterize LLMs entirely by input-output behaviour
- We know ***what*** LLMs do, but not ***why.***
- Despite all the performance improvements**, LLMs remain black boxes.**
- We can benefit from a **deeper understanding of their internal mechanisms**.

Key motivations

Safety & Trust — Detecting deception, hallucination, or misuse.
Accountability — Understanding *why* a model made a harmful or biased prediction.
Debugging & Model Editing — Fixing specific failure modes without retraining.
Scientific Curiosity — Reverse-engineering intelligence from emergent representations.
Regulation & Governance — EU AI Act and audit requirements depend on interpretability.

Four terms are often used interchangeably

- **Transparency is about the mechanism** — Opening the hood; understanding *how* each part contributes.
- **Explainability is about the audience** — Providing human-friendly explanations of outputs. An explanation can be persuasive without being faithful
- **Causality** — Understanding which components *cause* certain behaviors, as opposed to merely correlating with it.
- **Conceptual interpretability** asks whether internal features line up with human concepts such as negation, gender or sentiment.

Challenges of Interpreting **Large** Language Models

- **Scale:** 175B+ parameters; even a single attention head may encode thousands of sub-features.
- **Distributed representations:** No single neuron = single concept (superposition).
- **Depth & composition:** 100+ layers make causal attribution hard.
- **Emergent behavior:** New capabilities appear unpredictably with scale.
- **Dynamic context usage:** In-context learning means model “weights” aren’t the full story.

Timeline

2013–2016 — CNN Visualization Era

- *Zeiler & Fergus (2014)* — Deconv nets reveal early CNN features. Early layers detect edges and textures and later layers detect object parts
- *Olah et al. (2015–16)* — Feature visualization & neuron interpretability (Distill.pub).

  → Focus: ***seeing what neurons see.***

2018–2020 — Linguistic Probing in NLP

- *Alain & Bengio (2017)* — Linear probes for representations.
- *Tenney et al. (2019)* — BERT layers mirror NLP pipeline; from part-of-speech to syntax to semantics;
- *Hewitt & Manning (2019)* — Syntax geometry in embeddings.
- *Clark et al. (2019)* — Attention heads learn linguistic roles.

  → Focus: ***what information is encoded.***

2020–2022 — Mechanistic Interpretability

- *Elhage et al. (2021, Anthropic)* — Framework for transformer circuits.
- Olsson et al. (Anthropic, 2022) characterised induction heads, which underlie in-context copying
- *Nanda et al. (2022)* — Grokking analysis in modular arithmetic via circuits.
- *Wang et al. (2022, Anthropic)* — reverse-engineered the indirect-object-identification circuit in GPT-2

  → Focus: ***how transformers compute***.

2023–2024 — Sparse Features & Superposition

- *Anthropic (2023)* — Sparse Autoencoders (SAE) for monosemantic features.
- *Conmy et al. (2023)* — LLMs labeling their own features.
- *Nanda (2024)* — TransformerLens tools for large-scale analysis.

  → Focus: ***neurons -> features***

2023–2025 — Causal & Editable Models

- *Meng et al. (2023, ROME)* — Factual circuit editing.
- *Hernandez et al. (2023)* — Causal scrubbing methods gave a way to test circuit hypotheses; steering vectors and representation engineering
- *Zou et al. (2024)* — Steering vectors for controllable behavior.

  → Focus: ***intervention & control.***

2024–2025 — Automated & Scalable Interpretability

- *Anthropic (2024)* — Automated feature labeling via LLMs.
- *SAEBench, Neuronpedia (2024–25)* — Open interpretability datasets.

  → Focus: ***AI interpreting AI.***

Broad Taxonomy

Explanation scopes

Local Explanation

Why did the model produce this output for this input?

- Feature attribution
- Attention-based
- Example-based
- Natural language explanation

Global Explanation

What has the model learned in general, independent of any single input?

- Probing-based
- Neuron Activation Analysis
- Concept-based
- Mechanistic Interpretation

Local methods treat the model largely as a function and explain its outputs; global methods open it up and study its internal representations and computations

Based on Explainability for Large Language Models: A Survey

Local Explanation-based Analysis

Explain *why* the model produced a specific output

These methods are the most commonly used in practice, e.g., in auditing a single decision, because they do not require any understanding of the model's internals beyond access to outputs or gradients.

1. Feature Attribution

Compute contribution of each input token or feature to the output.

$\boldsymbol{x} = \{x_1, x_2, \ldots, x_n\}$ → LM (f) → $f(\boldsymbol{x})$

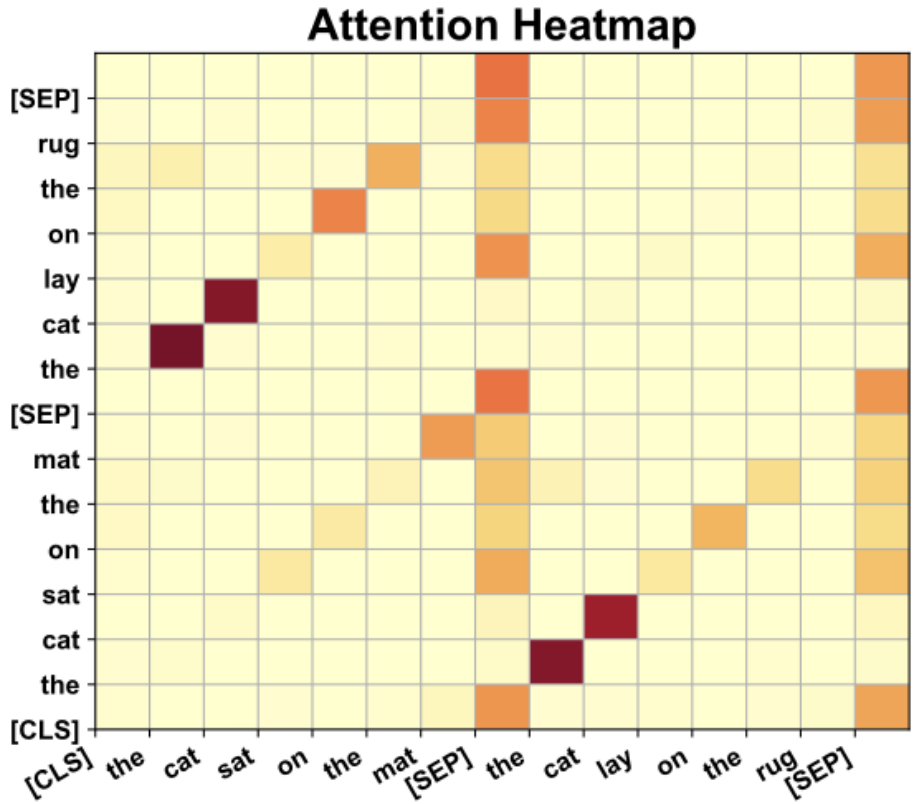
Assign relevance scores ($R(x_i)$)

Perturbation-Based	Gradient-Based	Surrogate Models	Decomposition Based
Perturb Input features, observe output, draw conclusions	Analyse the partial derivatives of the output wrt each input dimension	Use simpler, more human-comprehensible models to explain individual predictions of black-box models	Break down the relevance score into linear contributions from the input. [Layer-wise or end-to-end]
E.g. Leave-one-out	E.g. Integrated gradients (IG)	E.g. SHAP, LIME	E.g. Layer-wise relevance propagation (LRP), Taylor-type decomposition (DTD)

2. Attention-based Methods

Explain models solely based on the attention weights or by analyzing the knowledge encoded in the attention.

Visualizations	Function-Based	Probing Based
Visualizing attention heads for a single input using bipartite graphs or heatmaps	Attribution scores that blend attention and gradients generally perform better than using either alone, as they fuse more information.	Train classifiers on attention patterns to test whether heads encode syntax/coreference . This is closer to a global method,



(b) Heatmap

Is Attention Interpretable?

Sofia Serrano, Noah A. Smith

Attention mechanisms have recently boosted performance on a range of NLP tasks. Because attention layers explicitly weight input components' representations, it is also often assumed that attention can be used to identify information that models found important (e.g., specific contextualized word tokens). We test whether that assumption holds by manipulating attention weights in already-trained text classification models and analyzing the resulting differences in their predictions. While we observe some ways in which higher attention weights correlate with greater impact on model predictions, we also find many ways in which this does not hold, i.e., where gradient-based rankings of attention weights better predict their effects than their magnitudes. We conclude that while attention noisily predicts input components' overall importance to a model, it is by no means a fail-safe indicator.

Attention is not Explanation

Sarthak Jain, Byron C. Wallace

Attention mechanisms have seen wide adoption in neural NLP models. In addition to improving predictive performance, these are often touted as affording transparency: models equipped with attention provide a distribution over attended-to input units, and this is often presented (at least implicitly) as communicating the relative importance of inputs. However, it is unclear what relationship exists between attention weights and model outputs. In this work, we perform extensive experiments across a variety of NLP tasks that aim to assess the degree to which attention weights provide meaningful `explanations' for predictions. We find that they largely do not. For example, learned attention weights are frequently uncorrelated with gradient-based measures of feature importance, and one can identify very different attention distributions that nonetheless yield equivalent predictions. Our findings show that standard attention modules do not provide meaningful explanations and should not be treated as though they do. Code for all experiments is available at this https URL.

Attention is not not Explanation

Sarah Wiegreffe, Yuval Pinter

Attention mechanisms play a central role in NLP systems, especially within recurrent neural network (RNN) models. Recently, there has been increasing interest in whether or not the intermediate representations offered by these modules may be used to explain the reasoning for a model's prediction, and consequently reach insights regarding the model's decision-making process. A recent paper claims that `Attention is not Explanation' (Jain and Wallace, 2019). We challenge many of the assumptions underlying this work, arguing that such a claim depends on one's definition of explanation, and that testing it needs to take into account all elements of the model, using a rigorous experimental design. We propose four alternative tests to determine when/whether attention can be used as explanation: a simple uniform-weights baseline; a variance calibration based on multiple random seed runs; a diagnostic framework using frozen weights from pretrained models; and an end-to-end adversarial attention training protocol. Each allows for meaningful interpretation of attention mechanisms in RNN models. We show that even when reliable adversarial distributions can be found, they don't perform well on the simple diagnostic, indicating that prior work does not disprove the usefulness of attention mechanisms for explainability.

3. Example-based explanations

Identify **influential training or in-context examples**, by analysing changes in output when input is changed strategically.

Adversarial Examples	Counterfactual explanations	Data Influence
Carefully crafted modifications to the input can alter model decisions while barely being noticeable to humans.	Generated data point that is as close to the input data point as possible but for which the model gives a different outcome.	Characterizes the influence of individual training sample by measuring how much they affect the loss on test points.
For example, Wei et al. show that simply asking an LLM to begin its response with "Absolutely! Here's..." could mislead the model into complying with a harmful request.	For example, if a user was denied a loan by a machine learning model, an example counterfactual explanation could be: "Had your income been $5000 greater per year and your credit score been 30 points higher, your loan would be approved." [https://arxiv.org/pdf/1905.07857]	Kwon et al. propose DataInf https://arxiv.org/pdf/2310.00902

4. Natural Language Explanation

- **Explaining a model's decision making on an input sequence with generated text.**
- The basic approach for generating natural language explanations involves training a language model using both original textual data and human-annotated explanations.
- The trained language model can then automatically generate explanations in natural language.
- As explanations provide additional contextual space, they can improve downstream prediction accuracy and perform as a data augmentation technique.

4. Natural Language Explanation

- The important *caveat* is **faithfulness**.
- A generated explanation is another output of the model; it need not reflect the computation that actually produced the answer.
- Research on CoT faithfulness has shown models can give plausible rationales that omit the real reason, such as a bias in the prompt.
- So natural language explanations are excellent for plausibility and user communication, but weak as evidence about mechanism.
- This is precisely why we need the global and mechanistic methods that follow.

Global Explanation based Analysis

Reveal the *latent structure* of an LLM — its internal representations, circuits, and conceptual organization — instead of just explaining one decision.

Methods

- Linear Probing
- Sparse Autoencoder-based decomposition
- Logit Lens

Classifier-based Probing

Original model:

$$f : x \mapsto \hat{y}, \quad \mathcal{D}_O = \{(x^{(i)}, y^{(i)})\}, \quad \text{PERF}(f, \mathcal{D}_O)$$

Intermediate representations at layer l:

$$f_l(x)$$

The probing classifier is trained and evaluated on its own annotated dataset: $\mathcal{D}_P = \{(x^{(i)}, z^{(i)})\}$

Then we measure the classifier performance: $\text{PERF}(g, f, \mathcal{D}_O, \mathcal{D}_P)$

From an information-theoretic view, training the probe estimates the mutual information between the representations and the property, *I(z,h),* where *z* is a random variable over linguistic properties and *h* represents the intermediate representations $f_l(x)$.

Ref: Probing Classifiers: Promises, Shortcomings, and Advances

Classifier-based Probing - Limitation

- A main limitation of the probing classifier paradigm is the **disconnect between the probing classifier g and the original model f.**
- The probing framework may indicate **correlations** between representations $f_l(x)$ and linguistic property z, but it does not tell us whether this property is involved in **predictions** of f.
- In essence, correlations (with external properties) discovered by the probes are not appropriately indicative of the causality in the predictions of the original model.

Correlation ✔ Causation ✘

Neuron and Activation-level explanation

- Instead of examining the whole vector space, neuron analysis looks into individual dimensions, i.e., neurons in representations, that are crucial for model performance or associated with specific linguistic properties.

Concept-level Explanations

- Concept-based interpretability algorithms **map the inputs to a set of concepts** and measure **importance score of each pre-defined concept to model predictions**.
- By introducing abstract concepts, models can be explained in a human-understandable fashion rather than on low-level features. Information in latent space can also be transformed into comprehensible explanations.

Mechanistic Interpretability

- The field of study of **reverse engineering neural networks** from the learned weights down to **human-interpretable algorithms**.
- Analogous to reverse engineering a compiled program binary back to source code.
- Exposes ways to start controlling outputs of a model.

Key Concepts

- **Representation -** Every token → vector in a high-dimensional space.
  - contains linguistic, factual, and conceptual structure.
- **Feature -** a property of an input to the model, or some subset of that input (e.g., a token in the prompt given to a language model, or a patch of an image)
- **Concept -** human-meaningful abstraction — like “negation,” “politeness,” “gender" that can be represented as a direction or cluster in activation space.
- **Neuron -** A single scalar output in a neural network layer — the result of a linear transformation followed by a nonlinearity.
- **Residual stream** - The *residual stream* is the main communication channel that runs through all transformer layers, carrying forward the sum of previous activations.
  - $x_{l+1} = x_l + \text{Attention}(x_l) + \text{MLP}(x_l)$
  - Each layer *adds or edits* information without erasing what came before.
- **Circuit -** a causal subgraph of neurons, heads, or layers that collectively implement a specific computation.

Key Concepts

- **Linear Representation hypothesis -** features of the input can be represented as directions in activation space, and an activation is roughly a sum of feature directions scaled by how strongly each feature is present.

    - **Decomposability**: Network representations can be described in terms of independently understandable features.
    - **Linearity**: Features are represented by direction.

- To reverse engineer neural networks - we need decomposability.
- We also need to be able to 'access' the decomposition somehow.
    - Identify the individual features within a representation. In a **linear representation**, this corresponds to determining **which directions in activation space correspond to which independent features of the input**.

Key Concepts

- **Superposition**
  - Linear representations can represent more features than dimensions, using a strategy we call superposition. This can be seen as neural networks simulating larger networks.

  - This pushes features away from corresponding to neurons.

Suppose two linguistic features need to be encoded:

- F_1: *The sentence is a question.*
- F_2: *The subject is plural.*

Instead of dedicating one neuron for each, the model might store both features in overlapping patterns:

Neuron	Encodes combination of…
N_1	$0.7 \times F_1 + 0.3 \times F_2$
N_2	$-0.4 \times F_1 + 0.8 \times F_2$

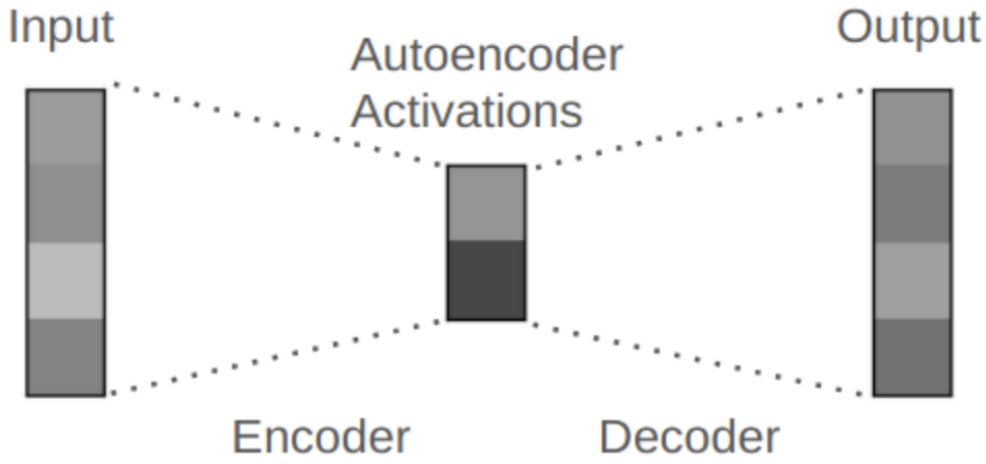
To recover each feature, the model applies **linear transformations** downstream to *untangle* the mixed signals — similar to unmixing sound waves.

Key Concepts

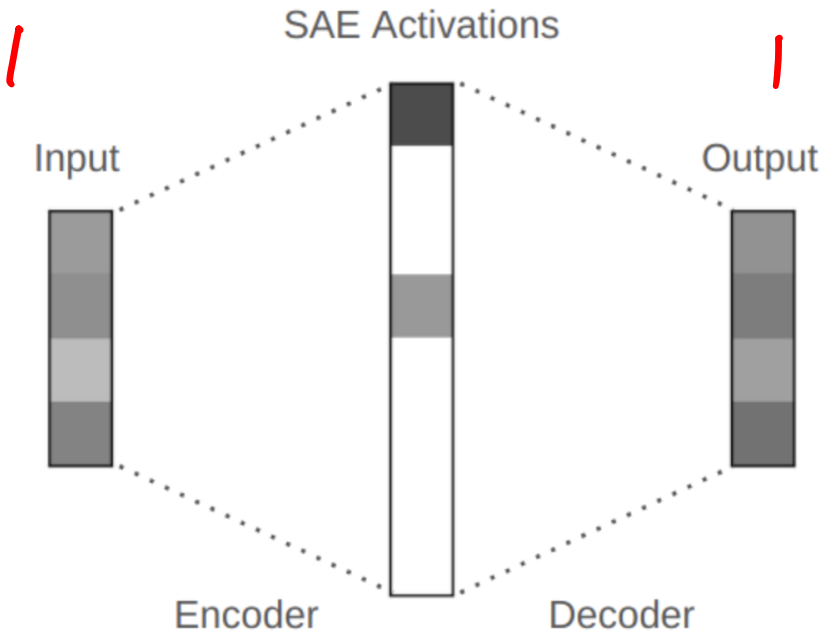
- **Superposition**
  - Linear representations can represent more features than dimensions, using a strategy we call superposition. This can be seen as neural networks simulating larger networks.
  - This pushes features away from corresponding to neurons.

- **Polysemanticity of Neurons:** Some neurons appear to respond to unrelated mixtures of inputs
  - vs monosemanticity - a neuron is **monosemantic** if it corresponds to a single feature.

Sparse Autoencoders



Normal Autoencoder

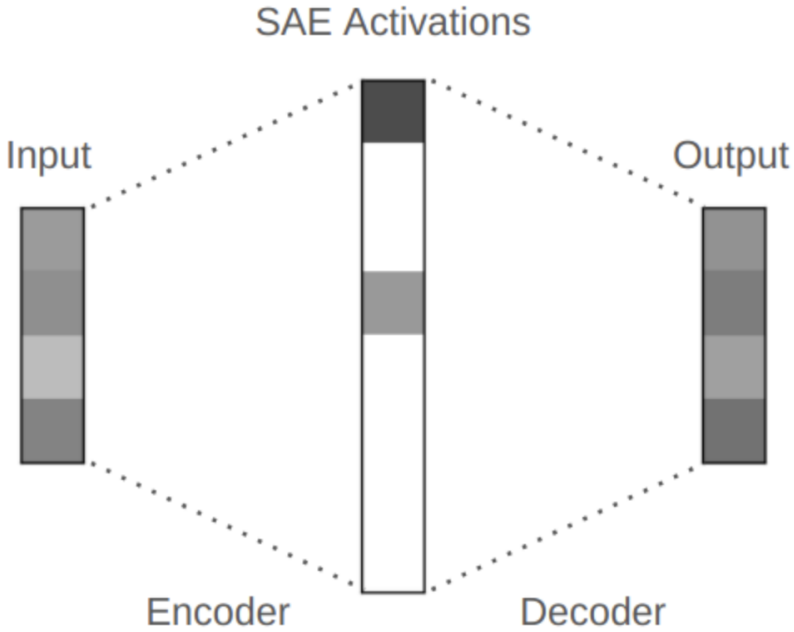


Sparse Autoencoder

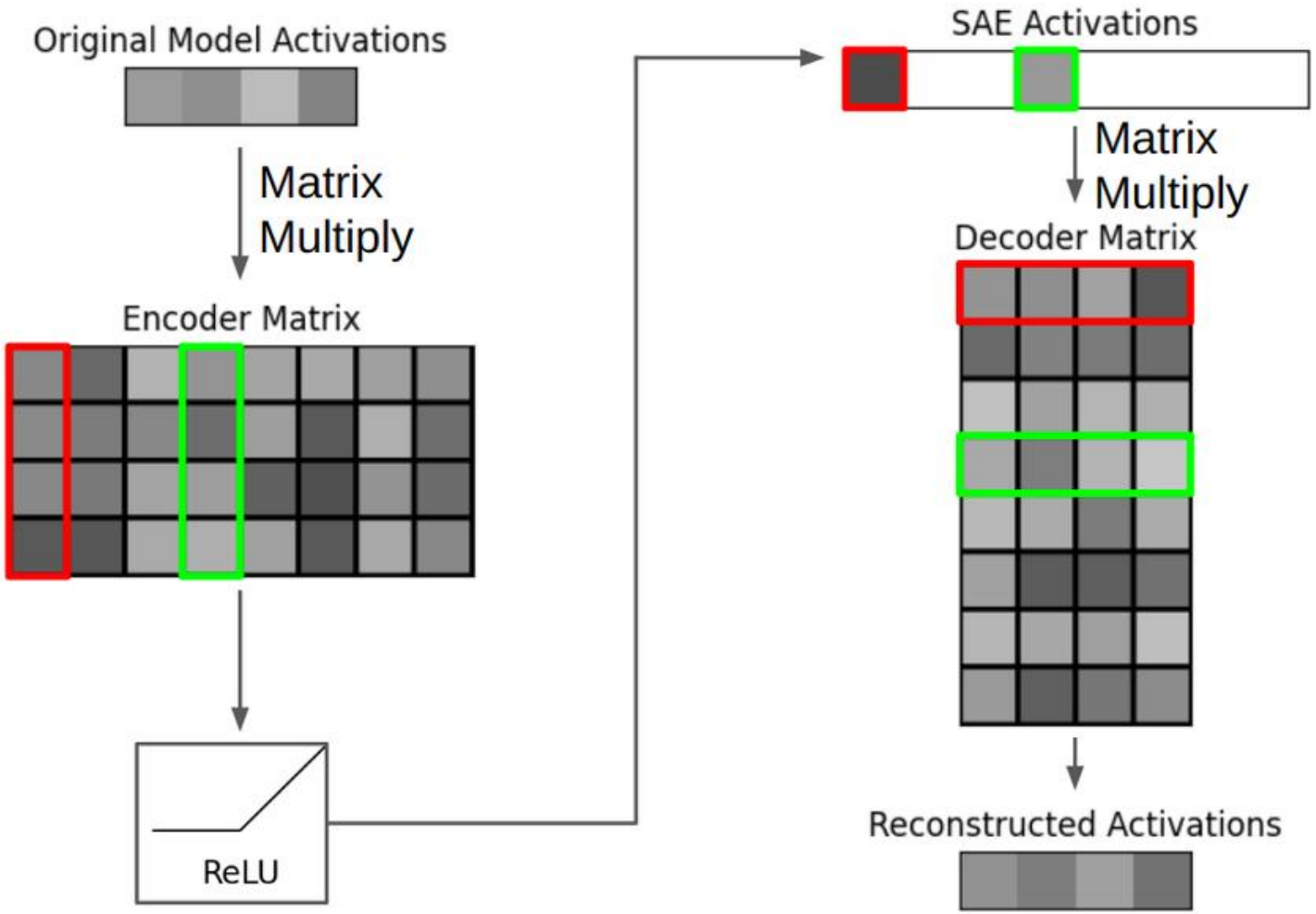
https://adamkarvonen.github.io/machine_learning/2024/06/11/sae-intuitions.html

Sparse Autoencoders

- Tool for discovering **interpretable features** in model activations, trying to mitigate **polysemanticity**.
- SAEs compress high-dimensional activations into a few active latent features. The **sparsity constraint** (enforced by loss function) forces the model to isolate independent, interpretable concepts.
- Each latent unit ≈ one semantically meaningful *feature*.



Sparse Autoencoders



Input encoding:

$$h = f_{\text{enc}}(a) = \text{ReLU}(W_E a + b_E)$$

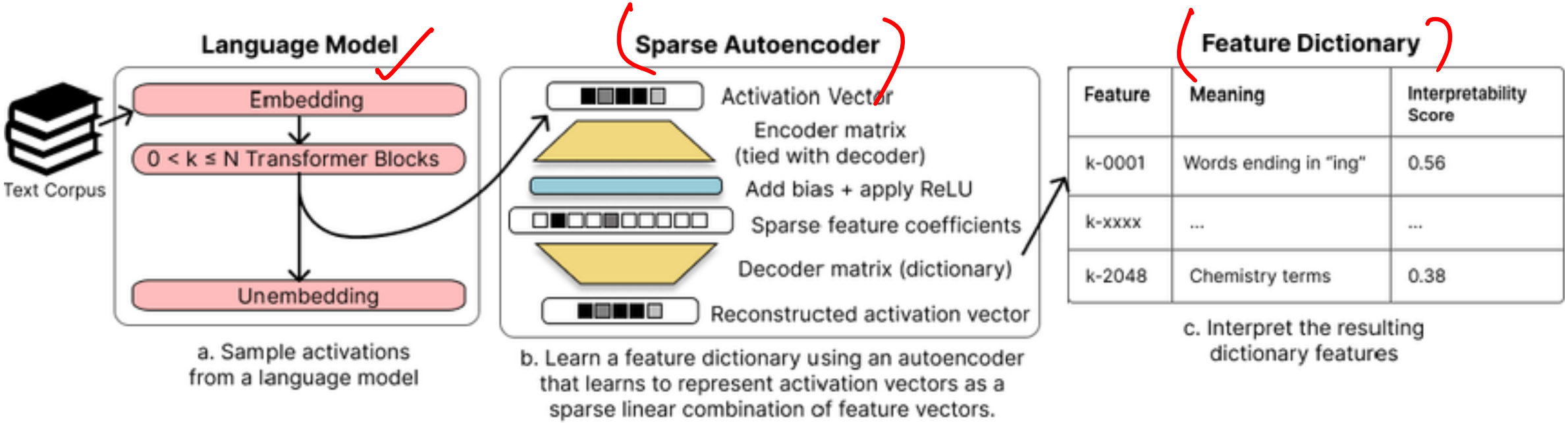
Decoding (reconstruction):

$$\hat{a} = f_{\text{dec}}(h) = W_D h + b_D$$

Loss function (reconstruction loss + sparsity loss):

$$\mathcal{L} = \|a - \hat{a}\|_2^2 + \lambda \|h\|_1$$

Sparse Autoencoders



The assumption is that each learnt column of the decoder matrix corresponds to an interpretable feature. Hence all the columns for a feature dictionary.

The inputs that maximise the column's corresponding latent activation reveal its semantics.

Ref: [Sparse Autoencoders Find Highly Interpretable Features In Language Models]

Sparse Autoencoders: Feature Dictionary

Step 1: Find what turns the feature on

- Run a large text corpus through the language model. At each token, pass the activation through the trained SAE.
- Each latent unit (say k-0001) now has an activation value on every token.
- Collect the text snippets where that latent fires hardest (its top-activating examples).

Step 2: Get the meaning (explain)

- Show those top snippets to a strong LLM (the "explainer", e.g. GPT-4), with the activation value marked on each token.
  - Example: *runn*[**9**]*ing*, *walk*[**8**]*ing*, *the*[0], *cat*[0]...
- Ask it: "What do these have in common?" It replies with a short label, e.g. **"Words ending in 'ing'"**. That label goes in the Meaning column.

Sparse Autoencoders: Feature Dictionary

Step 3: Get the interpretability score (simulate and compare)

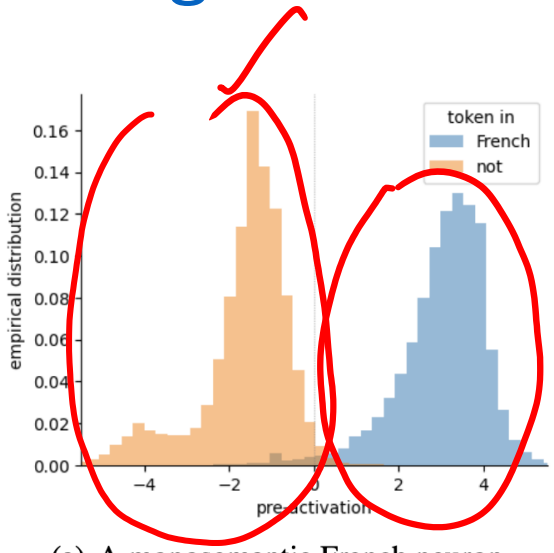
- The question now is whether the label really describes the feature.
- Give only the label to a second LLM (the "simulator"), together with new text it hasn't seen.
- For each token, the simulator predicts how strongly the feature should fire, based on the label alone. "Words ending in -ing", for example, means high on *jumping*, zero on *table*.
- Compare those predicted activations with the feature's real activations on the same text.
- **Score = correlation between predicted and real activations.**
  - Close to 1: the label predicts the feature's behaviour very well.
  - Close to 0: the label explains almost nothing.

Feature Dictionary

Feature	Meaning	Interpretability Score
k-0001	Words ending in "ing"	0.56
k-xxxx	...	...
k-2048	Chemistry terms	0.38

c. Interpret the resulting dictionary features

Finding neurons in a haystack – Case studies with Sparse Probing



(a) A monosemantic French neuron

Figure 1: Neurons that respond to single features (left) can be understood independently, in contrast to polysemantic neurons (right) that activate for many unrelated features (in this case, the occurrence of specific multi-token spans).

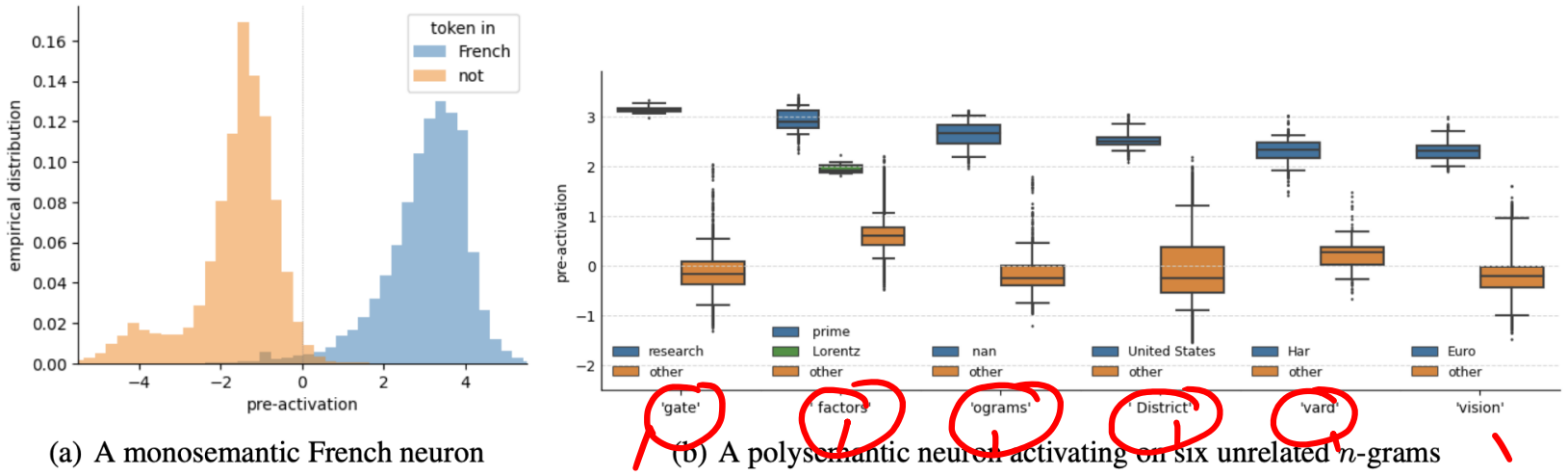
(c) Evidence of superposition in early layers

Some critical findings:

- Tremendous amount of interpretable structure within the neurons of LLMs but requires careful use and follow-up analysis to draw rigorous conclusions.
- The first 25% of fully connected layers employ substantially more superposition than the rest.
- As models increase in size, representation sparsity increases on average.

https://arxiv.org/pdf/2305.01610

Finding neurons in a haystack – Case studies with Sparse Probing



(a) A monosemantic French neuron

(b) A polysemantic neuron activating on six unrelated n-grams

Figure 1: Neurons that respond to single features (left) can be understood independently, in contrast to polysemantic neurons (right) that activate for many unrelated features (in this case, the occurrence of specific multi-token spans).

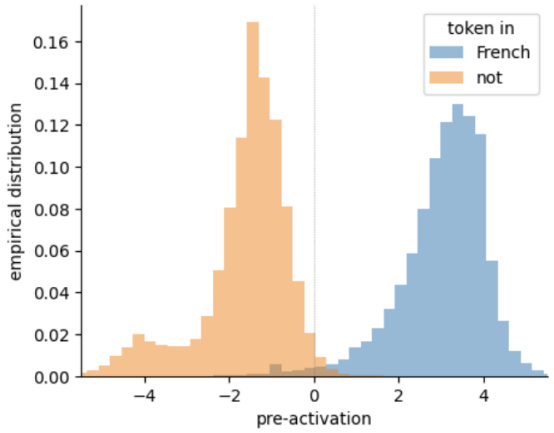
(c) Evidence of superposition in early layers

Some critical findings:

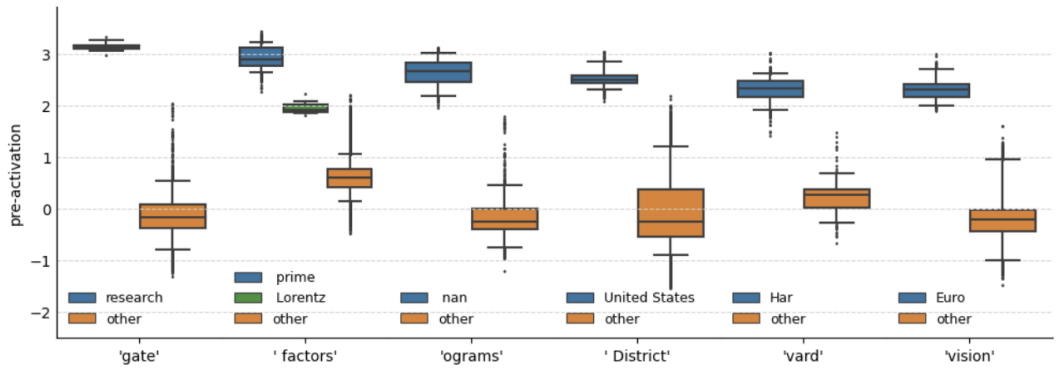
- Tremendous amount of interpretable structure within the neurons of LLMs but requires careful use and follow-up analysis to draw rigorous conclusions.
- The first 25% of fully connected layers employ substantially more superposition than the rest.
- As models increase in size, representation sparsity increases on average.

https://arxiv.org/pdf/2305.01610

Finding neurons in a haystack – Case studies with Sparse Probing

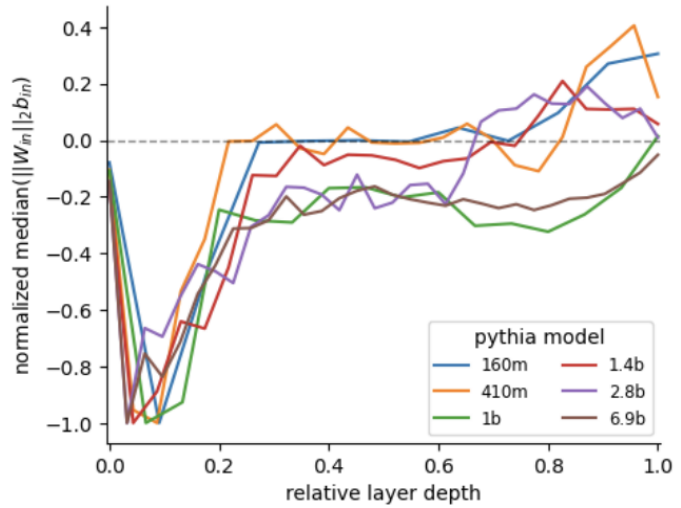


(a) A monosemantic French neuron



(b) A polysemantic neuron activating on six unrelated n-grams

Figure 1: Neurons that respond to single features (left) can be understood independently, in contrast to polysemantic neurons (right) that activate for many unrelated features (in this case, the occurrence of specific multi-token spans).



(c) Evidence of superposition in early layers

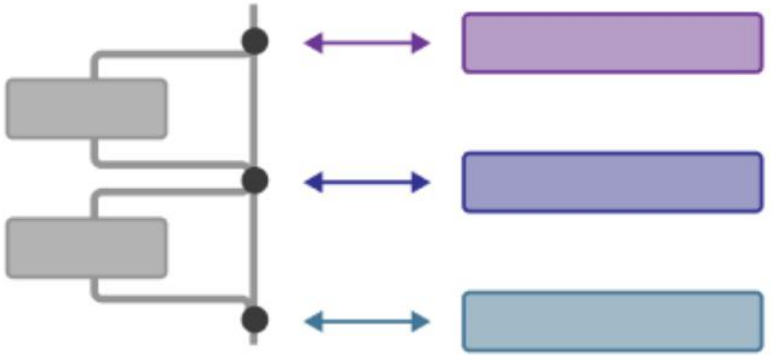
Some critical findings:

- Tremendous amount of interpretable structure within the neurons of LLMs but requires careful use and follow-up analysis to draw rigorous conclusions.
- The first 25% of fully connected layers employ substantially more superposition than the rest.
- As models increase in size, representation sparsity increases on average.

https://arxiv.org/pdf/2305.01610

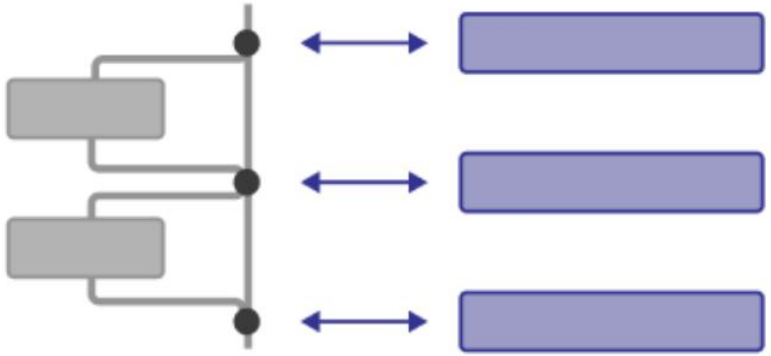
Sparse Autoencoders - Variations

Per-Layer SAEs



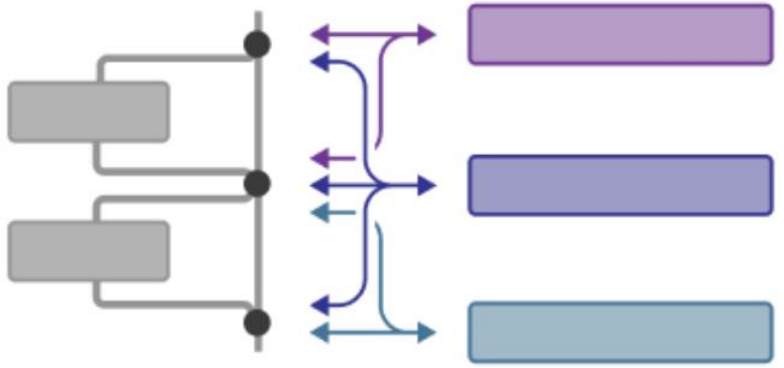
We train a different sparse autoencoder at each layer we wish to analyze. This is the "standard" approach at the moment.

Shared SAE



This approach (introduced by Yun et al.) has the same sparse autoencoder be used at every layer. This allows for a consistent notion of features across layers, with some features only existing at some layers. However, it doesn't allow features to rotate across layers (which we believe they sometimes do).

★ Sparse Crosscoder (our focus)

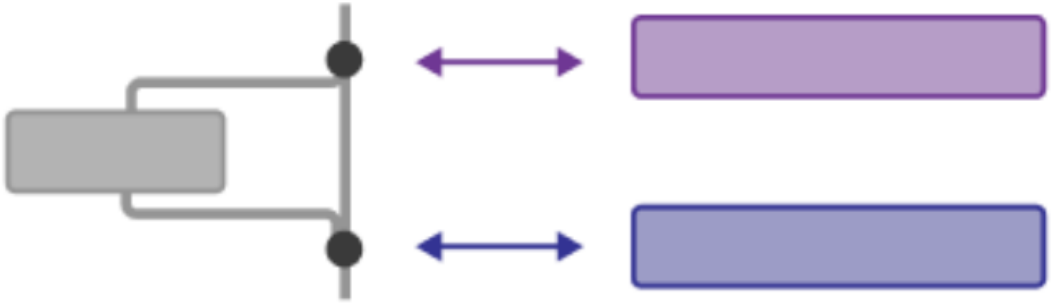


Sparse coders interact with multiple layers, resolving cross-layer superposition and tracking features through the residual stream. There may be one or multiple sparse autoencoders, and they may read and write to differing layers.

https://transformer-circuits.pub/2024/crosscoders/index.html

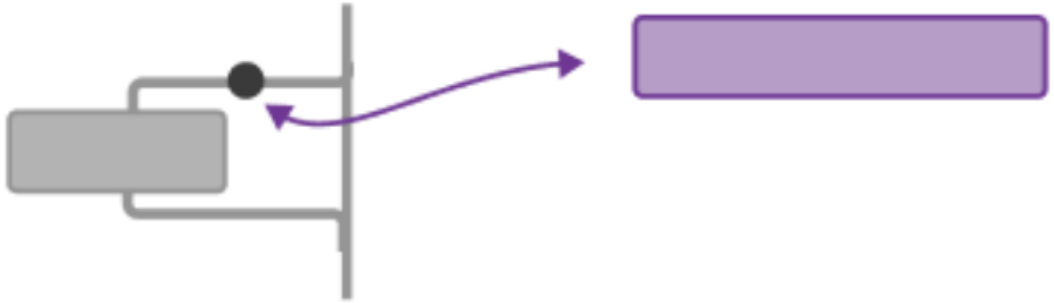
Sparse Autoencoders - Variations

Residual Stream



Sparse coder predict the residual stream.

Layer Outputs



Sparse coder predict the outputs of MLPs or attention before they're added to the residual stream. They may still read from the residual stream.

https://transformer-circuits.pub/2024/crosscoders/index.html

Transcoders vs SAEs

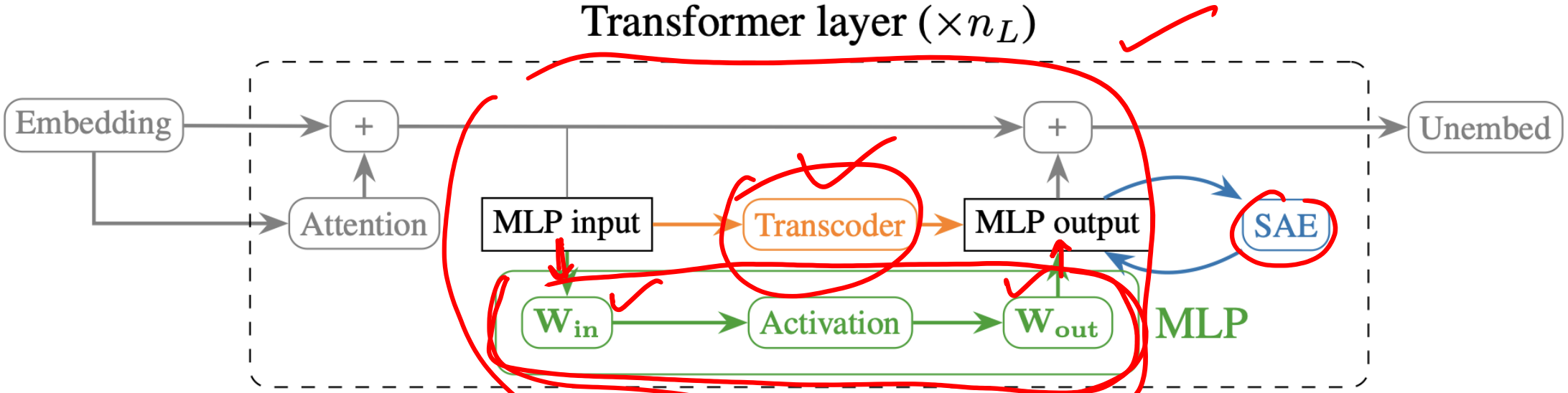


Figure 1: A comparison between SAEs, MLP transcoders, and MLP sublayers for a transformer-based language model. SAEs learn to reconstruct model activations, whereas transcoders imitate sublayers' input-output behavior.

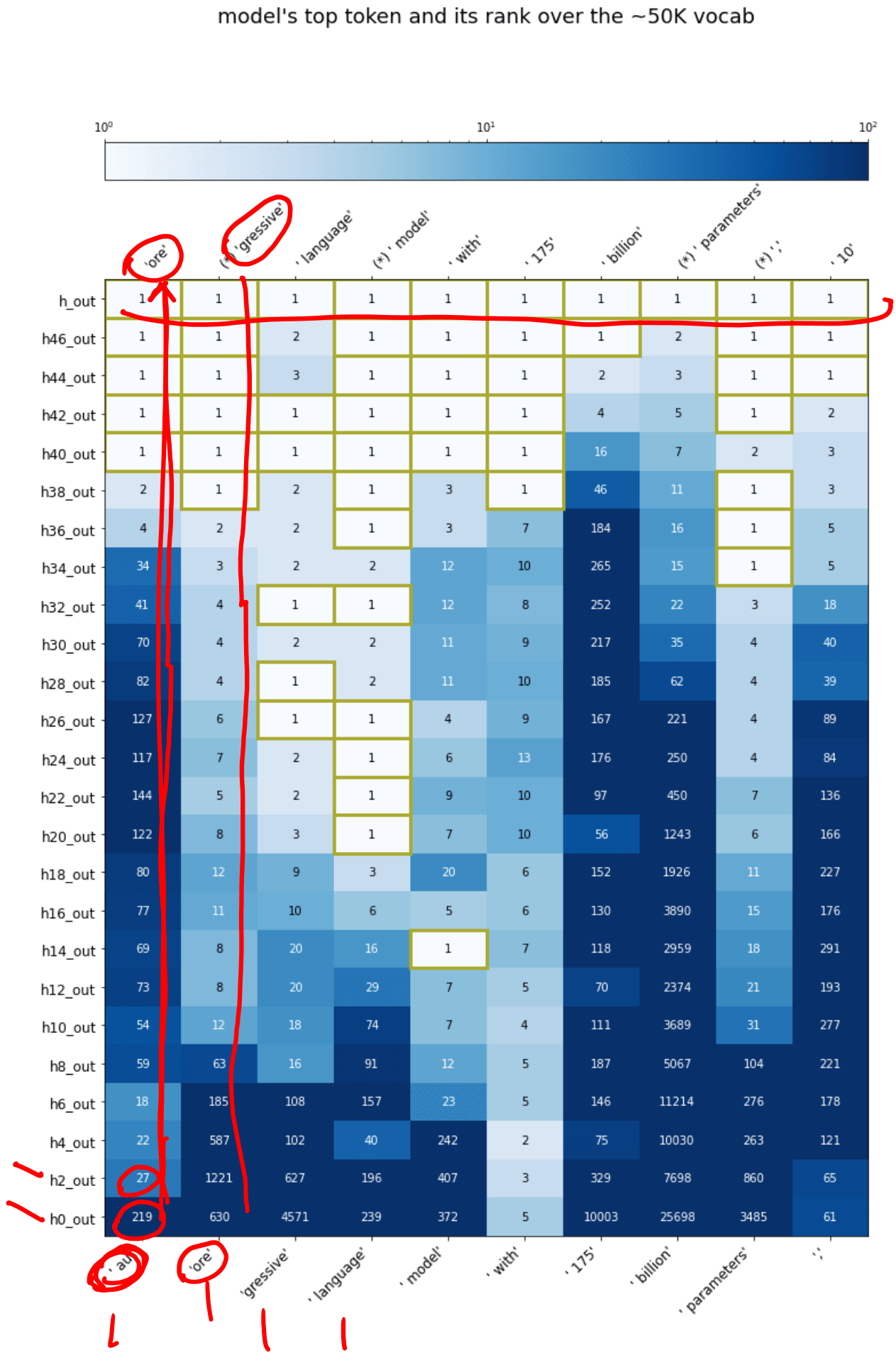
Ref: Transcoders find Interpretable LLM circuits

Logit Lens

- For analyzing **how predictions evolve** through model layers.
- Projects intermediate activations (residual stream) into **vocabulary space**.
- Shows which tokens the model "thinks" are likely at each layer.

$$\text{logits}_l = W_U \cdot a_l$$

- (where W_U is the unembedding matrix and a_l is the residual stream at layer l)



Tanmoy Chakraborty
Trustworthy LLMs

The Translation Bottleneck: PatchScopes

- **The Flaw in the Logit Lens**
  - The Unembedding Matrix (W_U) is strictly linear and was trained *only* to read the final layer.
  - Applying W_U to Layer 2 is like using a modern English dictionary to read ancient Latin. The information is there, but the translator doesn't understand the format.

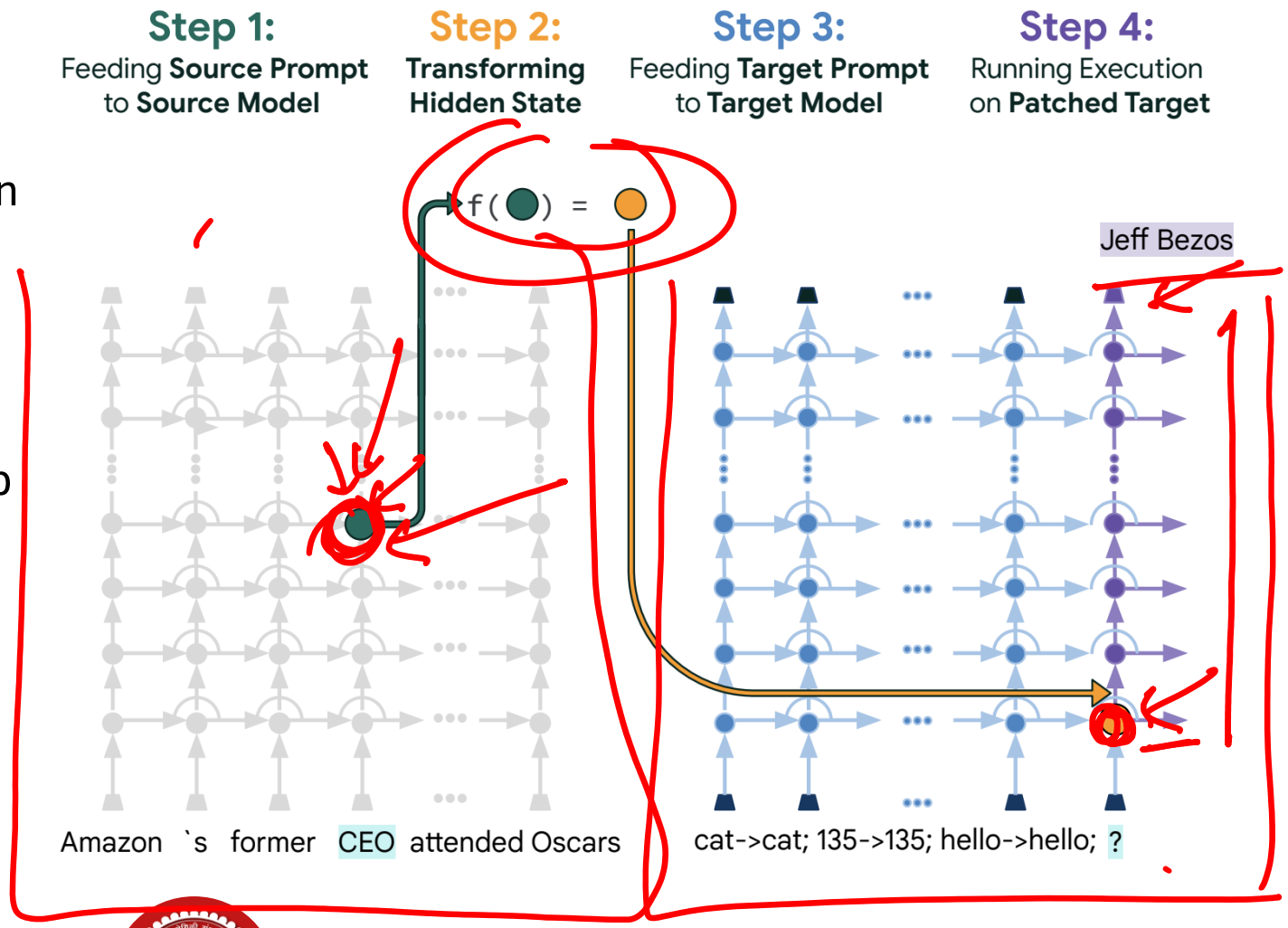
- **The Insight**
  - If W_U is a rigid dictionary, the Transformer's deep layers are a highly expressive, non-linear translation engine.

- **Patchscopes:**
  - Instead of forcing an early "thought" directly to the exit, what if we inject it into the later layers of a *different* prompt, and ask the model to explain it to us in natural language?

Patchscopes: The 4-Step Mechanism

The 4-Step Pipeline:

- **Source Execution:** Run the forward computation on the source prompt in the source model.
- **Transformation:** Optionally transform the hidden state at the source layer.
- **Target Execution:** Run the forward computation on the target prompt up to the target layer.
- **Patch & Decode:** Patch the target representation with the transformed source state and continue the forward computation to generate an explanation.



Ghandeharioun et. al, "Patchscopes: A Unifying Framework for Inspecting Hidden Representations of Language Models," *ICML 2024*

Patchscopes: Capabilities & Results

- **Early Layer Decoding:** Successfully translates early hidden states where the Logit Lens outputs gibberish.
- **Cross-Model Inspection:** We can use a highly capable model (e.g., GPT-4) as the "target" to explain the hidden states of a smaller, weaker "source" model.
- **Self-Correction:** Models can fix their own multi-hop reasoning errors by patching intermediate facts into new contexts.

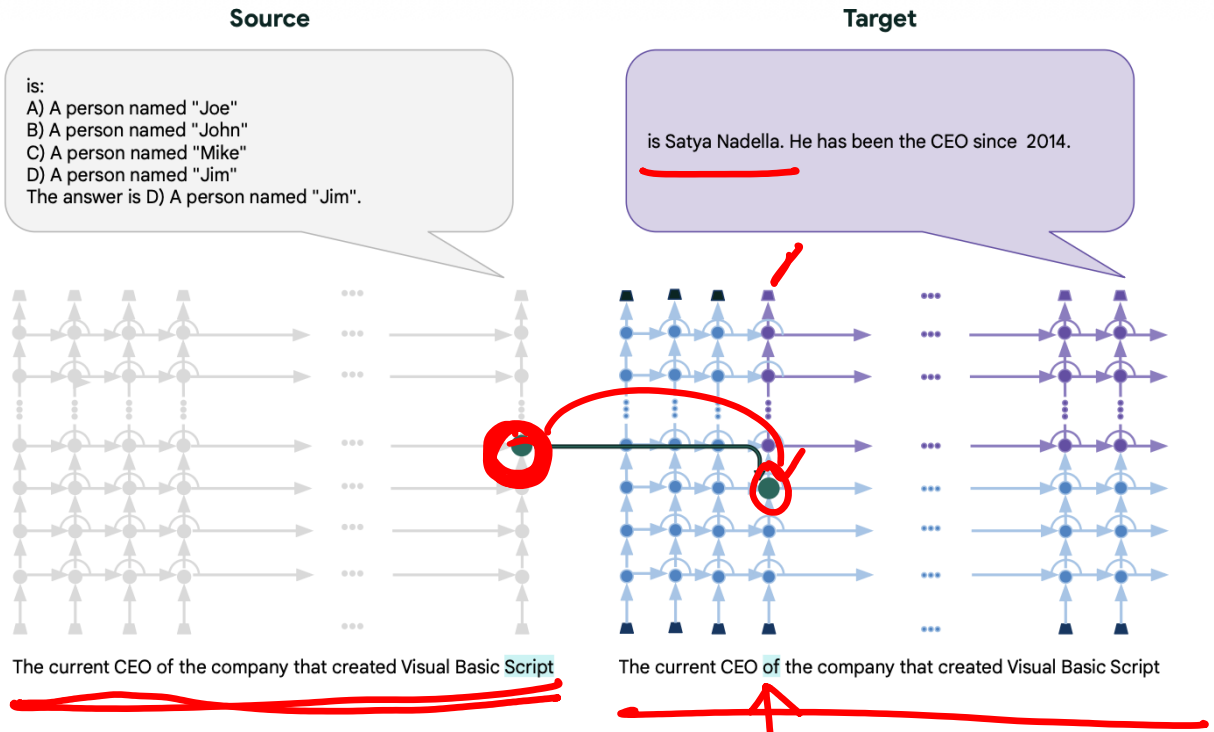


Figure 4. An illustration of CoT `Patchscope` on a single example. In this example, $\pi_1 \leftarrow$ "`the company that created Visual Basic Script`", $\pi_2 \leftarrow$ "`The current CEO of`", $S = T \leftarrow [\pi_2][\pi_1] =$ "`The current CEO of the company that created Visual Basic Script`". Note that $\mathcal{M} = \mathcal{M}^*$ and $f \leftarrow \mathbb{I}$.

Circuit Finding

- **Modular partitioning:** A post hoc way of understanding a deep NN in terms of modules is to partition the neurons into a set of subnetworks, each composed of related neurons. Toward this goal, some work divides neurons into modules based on graphical analysis of the network's weights and analyze how distinct the neurons in each module are. [https://arxiv.org/pdf/2207.13243]

- **Circuit Analysis:** Instead of analyzing an entire partition of a network, a much simpler approach is to study individual subnetworks inside of it. These have often been referred to as neural "circuits" which can be as small as just a few neurons and weights.

Activation Patching

- **Activation patching** (also called **causal tracing**): internal activations from one model run are "patched" into another run, enabling researchers to **causally probe** which components are necessary and sufficient for specific behaviors.

Standard activation patching procedure:

- Choose two prompts that differ in some key aspect:
  - Clean prompt (X_{clean}): elicits the desired behavior (e.g., "The Eiffel Tower is in" → "Paris")
  - Corrupted prompt ($X_{corrupt}$): differs in the dimension of interest (e.g., "The Colosseum is in" → "Rome")

- Run three model computations:
  - Clean run: Execute the model on X_{clean} and cache all activations
  - Corrupted run: Execute the model on $X_{corrupt}$ normally
  - Patched run: Execute the model on $X_{corrupt}$, but replace specific activations with those cached from the clean run

- Measure how the patched output changes using metrics like logit difference or KL divergence
- Repeat systematically across all components of interest (layers, attention heads, neurons)

Activation Patching

- **Key insight - activation patching reveals causal relationships:** if patching an activation from the clean run into the corrupted run restores the clean behavior, that component is **causally important** for the behavior. This allows us to identify which parts of the model encode specific information while **controlling for confounding factors**.

Correlation ✔ Causation ✔

Causal Tracing using activation patching – Example (Meng et al., 2022)

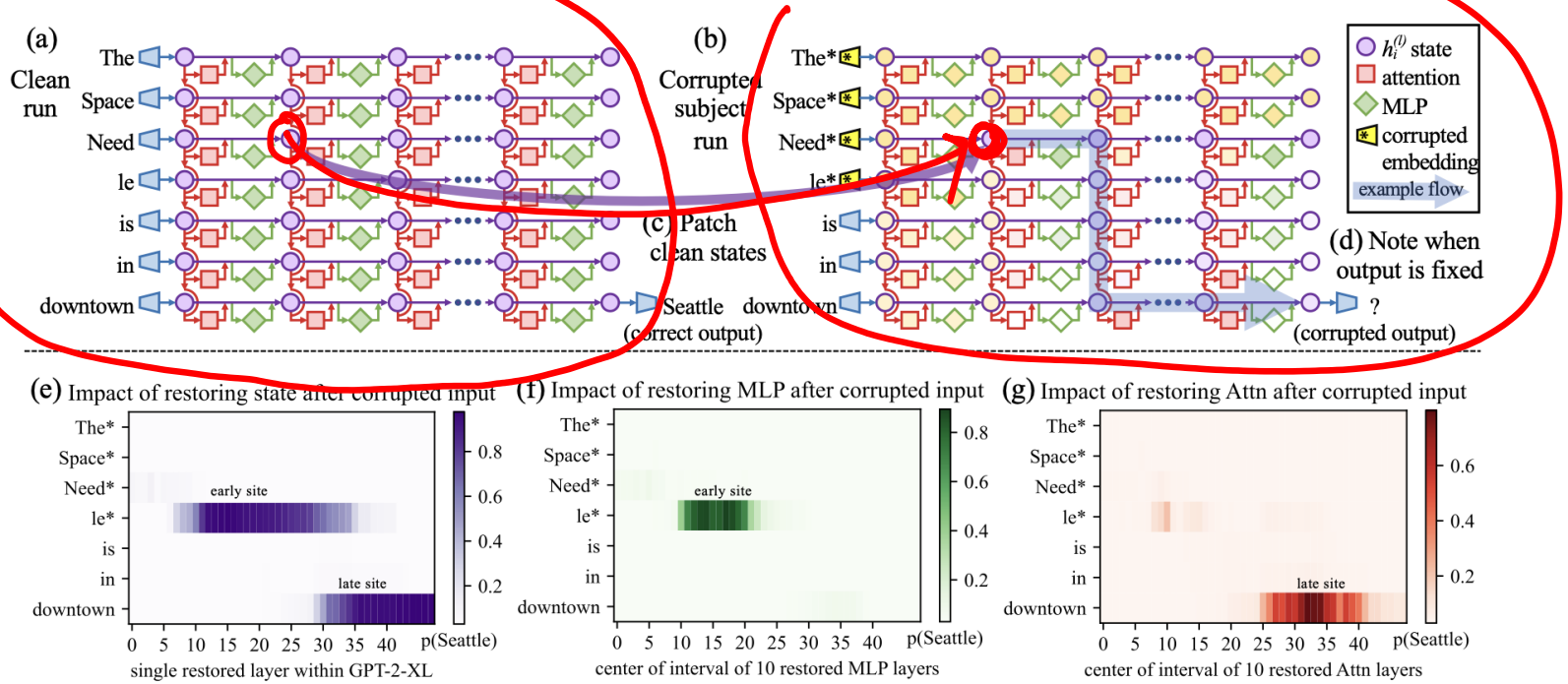


Figure 1: **Causal Traces** compute the causal effect of neuron activations by running the network twice: (a) once normally, and (b) once where we corrupt the subject token and then (c) restore selected internal activations to their clean value. (d) Some sets of activations cause the output to return to the original prediction; the light blue path shows an example of information flow. The causal impact on output probability is mapped for the effect of (e) each hidden state on the prediction, (f) only MLP activations, and (g) only attention activations.

Ref: ROME, a model editing method, uses causal mediation analysis to identify the specific modules that mediate recall of a fact about a subject

Activation Steering

- An **inference-time** technique for modifying LLM behavior by directly intervening on internal activations without retraining or changing model parameters. It works by adding vectorial modifications (called **steering vectors)** at specific layers in the model's computation to alter outputs along interpretable dimensions such as sentiment, truthfulness, style, or safety.

Steering Language Models With Activation Engineering

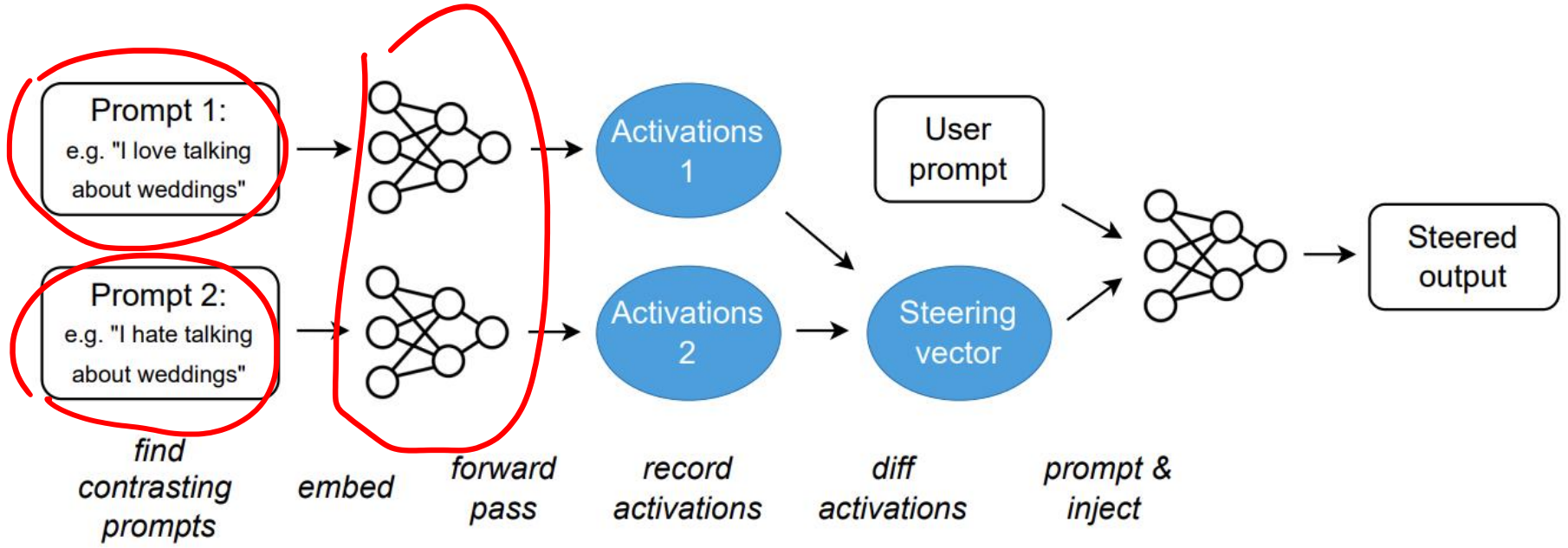


Figure 1: Schematic of the Activation Addition (**ActAdd**) method. ⬭ = natural language text; ● = vectors of activations just before a specified layer. In this example, the output is heavily biased towards discussing weddings, regardless of the topic of the user prompt. (See Algorithm 1 for the method's parameters: intervention strength, intervention layer, and sequence alignment.)

https://arxiv.org/pdf/2308.10248

Steering Language Models With Activation Engineering

Table 1: The impact of ActAdd. The steering vectors are computed from ("Love" - "Hate") and ("I talk about weddings constantly" - "I do not talk about weddings constantly"). Appendix Table 6 shows more examples.

Prompt	+	steering	=	completion
I hate you because...		[None]		...you are the most disgusting thing I have ever seen.
		ActAdd (love)		...you are so beautiful and I want to be with you forever.
I went up to my friend and said...		[None]		..."I'm sorry, I can't help you." "No," he said. "You're not."
		ActAdd (weddings)		..."I'm going to talk about the wedding in this episode of Wedding Season. I think it's a really good episode. It's about how you're supposed to talk about weddings."

Residual Stream Perspective of Transformers

Recall: Masked Self-Attention in Decoders

Self-Attention: Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

where, $Q = XW^Q$, $K = XW^K$, $V = XW^V$

Problem: While training autoregressive models, Transformers can see the future.

- For a current token x_i, the attention scores are computed with all tokens in the sequence including those which comes after x_i (as the whole sequence is available to us during training).

Solution: Masking

Recall: Masked Self-Attention in Decoders

Masking: 'Masked' scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where, masking matrix M is defined as:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

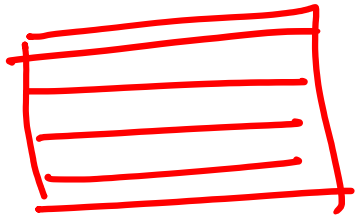
For future tokens, the attention scores becomes zero after applying softmax [*softmax(-∞) = 0*].

- Effectively, **after masking**, the query is the current token x_i, and the keys and values comes from the tokens before it, including itself (i.e., $x_j,\ j \leq i$).

N_E

N_E

$x_{\leq i}^{l}$



i^{th} position

$$a_i^{l,h} = \text{Softmax}\left(\frac{x_i^{l-1} W_Q^{l,h} \left(x_{\leq i}^{l-1} W_K^{l,h}\right)^T}{\sqrt{d_h}}\right)$$

$$= \text{softmax}\left(\frac{x_i^{l-1} W_Q^{l,h} \left(W_K^{l,h}\right)^T \left(x_{\leq i}^{l-1}\right)^T}{\sqrt{d_h}}\right)$$

$W_{QK}^{l,h} \in \mathbb{R}^{d \times d}$ | Query-Key circuit

$= W_Q^{l,h} \left(W_K^{l,h}\right)^T$

Re-writing the Masked Self-Attention Equation

Now let's re-write the masked attention equation for a current token x_i.

- Assume that we are considering the **attention head *h*** of **layer *l***.
- Let's denote the matrix with the **output hidden representation in layer *k* of previous tokens** x_j, $j \leq i$ as $X^k_{\leq i}$.

Thus, for calculating attention scores for **attention head *h*** of **layer *l***, input to the attention sub-layer is the output representation from the previous layer *l-1*.

- **Query:** $x_i^{l-1} W_Q^{l,h}$
- **Keys:** $X_{\leq i}^{l-1} W_K^{l,h}$

$$\boldsymbol{a}_i^{l,h} = \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1}\boldsymbol{W}_Q^{l,h}\,(\boldsymbol{X}_{\leq i}^{l-1}\boldsymbol{W}_K^{l,h})^\intercal}{\sqrt{d_k}}\right)$$

Query vector

Key vector

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

QK Circuit

$$\boldsymbol{a}_i^{l,h} = \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1}\boldsymbol{W}_Q^{l,h}\,(\boldsymbol{X}_{\leq i}^{l-1}\boldsymbol{W}_K^{l,h})^\intercal}{\sqrt{d_k}}\right)$$

Query vector

Key vector

$$= \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1}\boldsymbol{W}_{QK}^{h}\boldsymbol{X}_{\leq i}^{l-1\intercal}}{\sqrt{d_k}}\right),$$

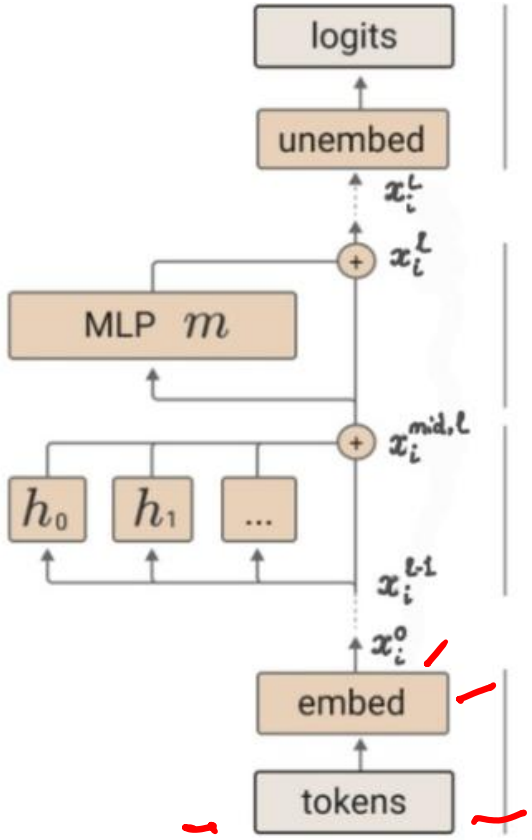
QK (query-key) circuit: $W_{QK}^h = W_Q^h W_K^{h\intercal}$

- QK circuits are responsible for reading from the **residual stream**.

Let's now look at the residual stream

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

Residual Stream Perspective



The final logits are produced by applying the unembedding.

$$T(t) = W_U x_i^L$$

An MLP layer, m, is run and added to the residual stream.

$$x_i^l = x_i^{mid,l} + m(x_i^{mid,l})$$

Each attention head, h, is run and added to the residual stream.

$$x_i^{mid,l} = x_i^{l-1} + \sum_{h=1}^{H} Attn^{l,h}(X_{\leq i}^{l-1})$$

One residual block

Token embedding.

$$x_i^0 = W_E t_i$$

Elhage, et al., A Mathematical Framework for Transformer Circuits

- Each input embedding gets updated via vector additions from the attention and feed-forward blocks producing **residual stream states** (or intermediate representations).
- The final layer residual stream state is then projected into the vocabulary space via the unembedding matrix $W_U \in R^{d \times |V|}$ and normalized via the *softmax*.

Combining the Output of Multiple Attention Heads

Query vector

Key vector

$$\boldsymbol{a}_i^{l,h} = \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1}\boldsymbol{W}_Q^{l,h}\,(\boldsymbol{X}_{\leq i}^{l-1}\boldsymbol{W}_K^{l,h})^\intercal}{\sqrt{d_k}}\right)$$

$$= \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1}\boldsymbol{W}_{QK}^{h}\boldsymbol{X}_{\leq i}^{l-1\intercal}}{\sqrt{d_k}}\right),$$

Value vector

$$\text{Attn}^{l,h}(\boldsymbol{X}_{\leq i}^{l-1}) = \sum_{j\leq i} a_{i,j}^{l,h}\,\boldsymbol{x}_j^{l-1}\boldsymbol{W}_V^{l,h}\boldsymbol{W}_O^{l,h}$$

$$= \sum_{j\leq i} a_{i,j}^{l,h}\,\boldsymbol{x}_j^{l-1}\boldsymbol{W}_{OV}^{l,h},$$

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

OV Circuit

$$\text{Attn}^{l,h}(\boldsymbol{X}_{\leq i}^{l-1}) = \sum_{j \leq i} a_{i,j}^{l,h} \boldsymbol{x}_j^{l-1} \boldsymbol{W}_V^{l,h} \boldsymbol{W}_O^{l,h}$$

Value vector

$$= \sum_{j \leq i} a_{i,j}^{l,h} \boldsymbol{x}_j^{l-1} \boldsymbol{W}_{OV}^{l,h}$$

OV (output-value) circuit: $W_{OV}^{l,h} = W_V^{l,h} W_O^{l,h}$

- OV circuits are responsible for writing to the **residual stream**.

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

Attention Block Output

The attention block output is the sum of individual attention heads, which is subsequently added back into the residual stream.

$$\text{Attn}^{l,h}(\boldsymbol{X}_{\leq i}^{l-1}) = \sum_{j \leq i} a_{i,j}^{l,h} \boldsymbol{x}_j^{l-1} \boldsymbol{W}_V^{l,h} \boldsymbol{W}_O^{l,h} \quad \text{(Value vector)}$$
$$= \sum_{j \leq i} a_{i,j}^{l,h} \boldsymbol{x}_j^{l-1} \boldsymbol{W}_{OV}^{l,h},$$

$$\boldsymbol{a}_i^{l,h} = \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1} \boldsymbol{W}_Q^{l,h} (\boldsymbol{X}_{\leq i}^{l-1} \boldsymbol{W}_K^{l,h})^\intercal}{\sqrt{d_k}} \right) \quad \text{(Query vector, Key vector)}$$
$$= \text{softmax}\left(\frac{\boldsymbol{x}_i^{l-1} \boldsymbol{W}_{QK}^{h} \boldsymbol{X}_{\leq i}^{l-1\intercal}}{\sqrt{d_k}} \right),$$

$$\text{Attn}^{l}(\boldsymbol{X}_{\leq i}^{l-1}) = \sum_{h=1}^{H} \text{Attn}^{l,h}(\boldsymbol{X}_{\leq i}^{l-1})$$

$$\boldsymbol{x}_i^{\text{mid},l} = \boldsymbol{x}_i^{l-1} + \text{Attn}^{l}(\boldsymbol{X}_{\leq i}^{l-1}).$$

$\boldsymbol{x}_i^{\text{mid},l}$

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

$$g\left(x_i^{mid,L} \cdot W_{in}^{L}\right) \cdot W_{out}^{L}$$



Feed-Forward Network (FFN)

$$\boldsymbol{x}_i^{\text{mid},l} = \boldsymbol{x}_i^{l-1} + \text{Attn}^l(\boldsymbol{X}_{\leq i}^{l-1}).$$

$$\text{FFN}^l(\boldsymbol{x}_i^{\text{mid},l}) = g(\boldsymbol{x}_i^{\text{mid},l}\boldsymbol{W}_{\text{in}}^l)\boldsymbol{W}_{\text{out}}^l.$$

$$\boldsymbol{x}_i^l = \boldsymbol{x}_i^{\text{mid},l} + \text{FFN}^l(\boldsymbol{x}_i^{\text{mid},l}).$$

$$\boldsymbol{W}_{\text{in}}^l \in \mathbb{R}^{d \times d_{\text{ffn}}}$$

$$\boldsymbol{W}_{\text{out}}^l \in \mathbb{R}^{d_{\text{ffn}} \times d}$$

- W_{in}^l reads from the residual stream state $x_i^{mid,l}$.
- Its result is passed through an element-wise non-linear activation function g, producing the neuron activations.
- These get transformed by W_{out}^l to produce the output $FFN^l(x_i^{mid,l})$, which is then added back to the residual stream

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

Key-value Memory Retrieval

$$\mathrm{FFN}^l(\boldsymbol{x}_i^{\mathrm{mid},l}) = g(\boldsymbol{x}_i^{\mathrm{mid},l}\boldsymbol{W}_{\mathrm{in}}^l)\boldsymbol{W}_{\mathrm{out}}^l.$$

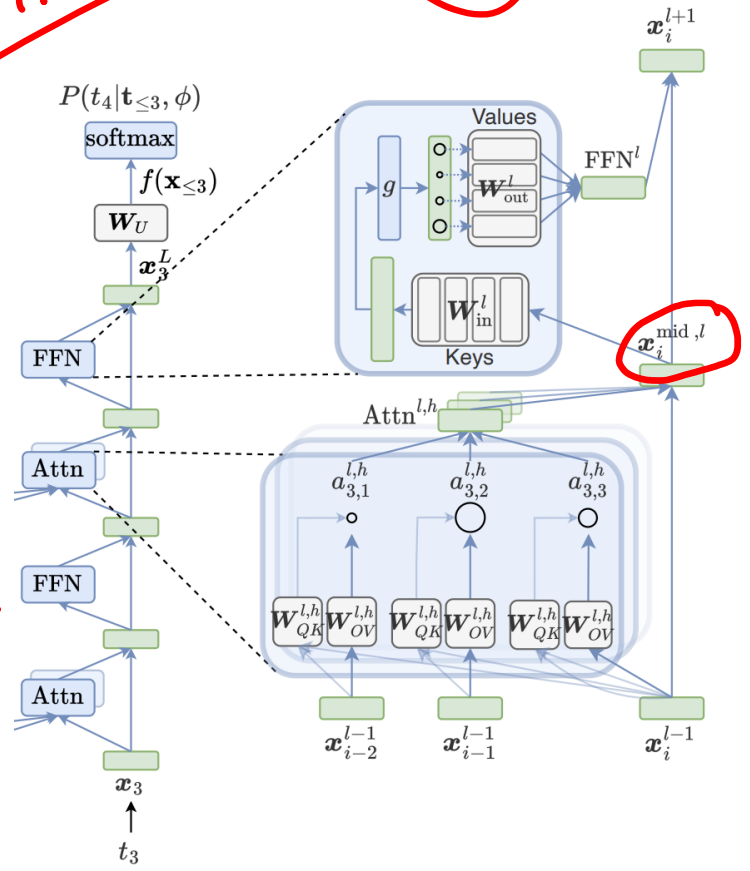
Keys (w^l_{in}) stored in columns of W^l_{in} act as pattern detectors over the input sequence

Values w^l_{out}, rows of W^l_{in}, are upweighted by each neuron activation

$$\mathrm{FFN}^l(\boldsymbol{x}_i^{\mathrm{mid},l}) = \sum_{u=1}^{d_{\mathrm{ffn}}} g_u(\boldsymbol{x}_i^{\mathrm{mid},l}\boldsymbol{w}_{\mathrm{in}_u}^l)\boldsymbol{w}_{\mathrm{out}_u}^l$$

$$= \sum_{u=1}^{d_{\mathrm{ffn}}} n_u^l \boldsymbol{w}_{\mathrm{out}_u}^l,$$

with $\boldsymbol{n}^l \in \mathbb{R}^{d_{\mathrm{ffn}}}$ being the vector of neuron activations, and n_u^l the u-th neuron activation value.



Prediction as a Sum of Component Outputs

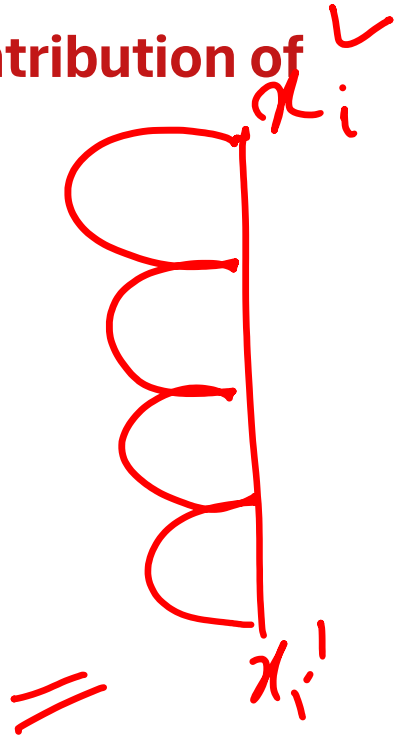
- Prediction head of a Transformer consists of an unembedding matrix: $\boldsymbol{W}_U \in \mathbb{R}^{d \times |\mathcal{V}|}$

We can rearrange the traditional forward pass formulation to separate the **contribution of each model component to the output logits**:

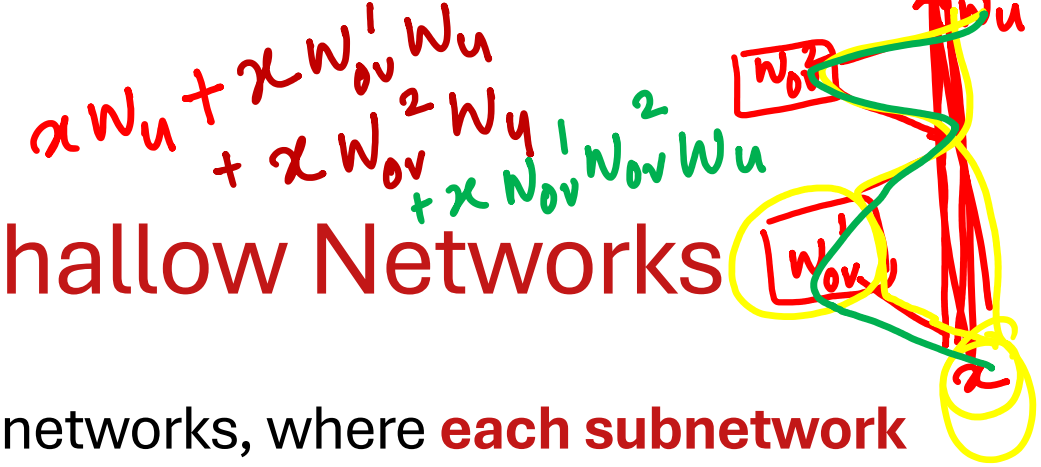
$$
\begin{aligned}
f(\mathbf{x}) &= \boldsymbol{x}_n^L \boldsymbol{W}_U \\
&= \left(\sum_{l=1}^{L} \sum_{h=1}^{H} \text{Attn}^{l,h}(\boldsymbol{X}_{\leq n}^{l-1}) + \sum_{l=1}^{L} \text{FFN}^{l}(\boldsymbol{x}_n^{\text{mid},l}) + \boldsymbol{x}_n \right) \boldsymbol{W}_U \\
&= \sum_{l=1}^{L} \sum_{h=1}^{H} \text{Attn}^{l,h}(\boldsymbol{X}_{\leq n}^{l-1}) \boldsymbol{W}_U + \sum_{l=1}^{L} \text{FFN}^{l}(\boldsymbol{x}_n^{\text{mid},l}) \boldsymbol{W}_U + \boldsymbol{x}_n \boldsymbol{W}_U .
\end{aligned}
$$

Attention head logits update

FFN logits update



Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models



Prediction as an Ensemble of Shallow Networks

- Residual networks work as ensembles of shallow networks, where **each subnetwork defines a path in the computational graph**.

Consider a two-layer attention-only Transformer, where each attention head is composed just by an OV matrix:

$$f(\boldsymbol{x}) = \boldsymbol{x}^1 + \boldsymbol{W}^2_{OV}(\boldsymbol{x}^1), \text{ with } \boldsymbol{x}^1 = \boldsymbol{x} + \boldsymbol{W}^1_{OV}(\boldsymbol{x})$$

We can decompose the forward pass as:

$$f(\boldsymbol{x}) = \underbrace{\boldsymbol{x}\boldsymbol{W}_U}_{\text{Direct path}} + \underbrace{\boldsymbol{x}\boldsymbol{W}^1_{OV}\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}^1_{OV}\boldsymbol{W}^2_{OV}\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}^2_{OV}\boldsymbol{W}_U}_{\text{Full OV circuits}}.$$

$\boldsymbol{x}\boldsymbol{W}^1_{OV}\boldsymbol{W}^2_{OV}\boldsymbol{W}_U$: Virtual attention heads (V-composition)

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

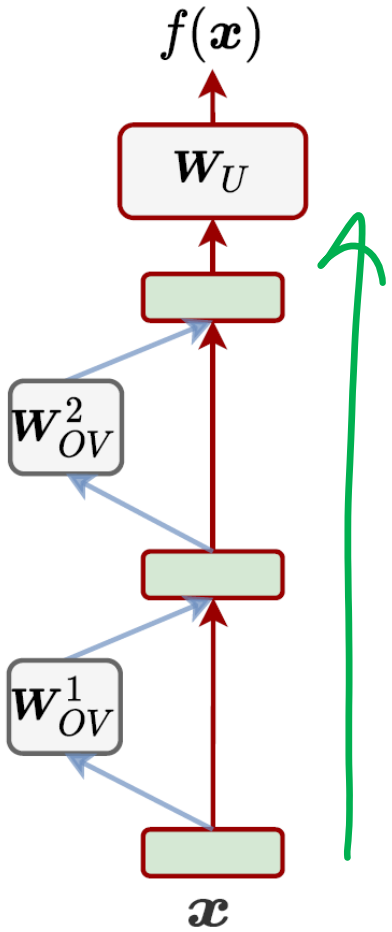
Prediction as an Ensemble of Shallow Networks

Direct path

Full OV circuits

$$f(\boldsymbol{x}) = \boldsymbol{x}\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^1\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^1\boldsymbol{W}_{OV}^2\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^2\boldsymbol{W}_U .$$

Virtual attention heads (V-compositior



- This term links the input embedding to the unembedding matrix and is referred to as the **direct path**
- It shows the contribution of the input embedding towards the output logit of the next token to be predicted.

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

Prediction as an Ensemble of Shallow Networks

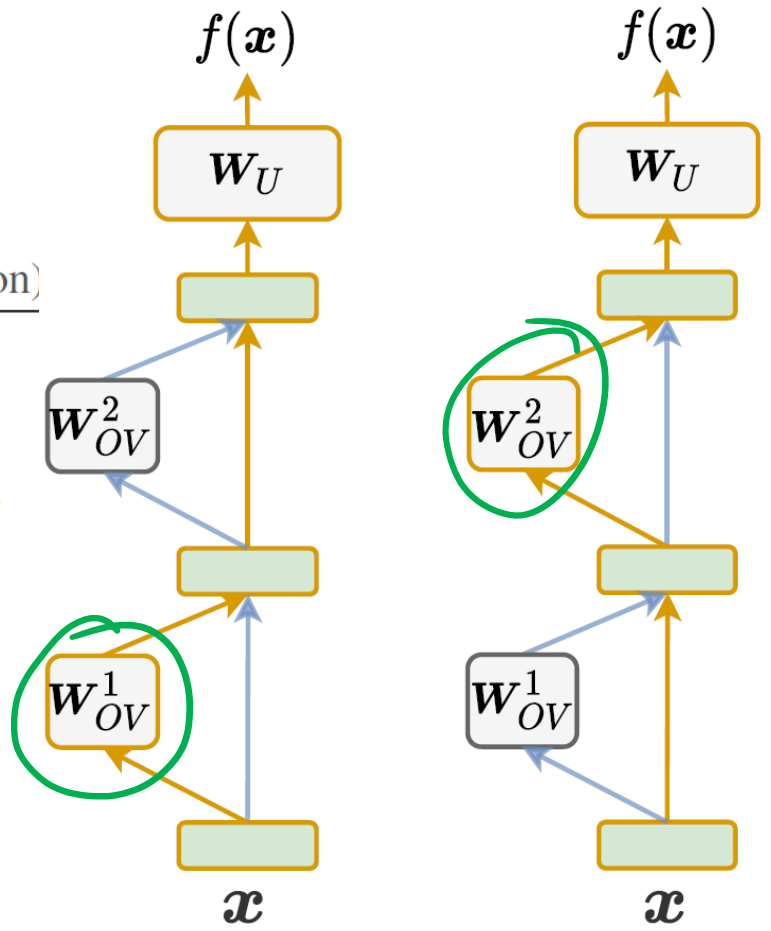
Direct path

Full OV circuits

$$f(x) = xW_U + xW_{OV}^1 W_U + xW_{OV}^1 W_{OV}^2 W_U + xW_{OV}^2 W_U .$$

Virtual attention heads (V-composition)

- These terms depicts paths traversing a single OV circuit, and are named **full OV circuits**
- They show the contribution of each OV circuit towards the output logit of the next token to be predicted.



Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

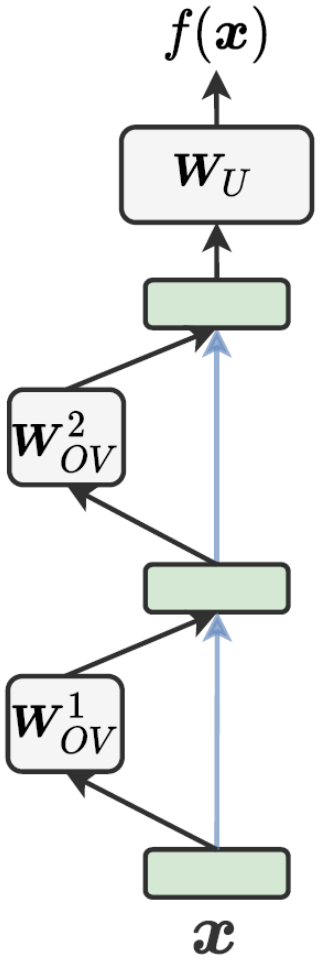
Prediction as an Ensemble of Shallow Networks

Direct path

Full OV circuits

$$f(\boldsymbol{x}) = \boldsymbol{x}\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^1\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^1\boldsymbol{W}_{OV}^2\boldsymbol{W}_U + \boldsymbol{x}\boldsymbol{W}_{OV}^2\boldsymbol{W}_U .$$

Virtual attention heads (V-composition)



- This term depicts the path involving both attention heads, and is referred to as **virtual attention heads doing V-composition**
- This is called 'composition' since the sequential writing and reading of the two heads is seen as OV matrices composing together.
  - The amount of composition can be measured as:

$$||W_{OV}^1 W_{OV}^2||_F \Big/ ||W_{OV}^1||_F ||W_{OV}^2||_F$$

Ferrando et al., A Primer on the Inner Workings of Transformer-based Language Models

Prediction as an Ensemble of Shallow Networks

- In full Transformer models, Q-composition and K-composition, i.e. compositions of W_Q and W_K with the W_{OV} output of previous layers, can also be found.

- Such decomposition enables us to localize the inputs or model components responsible for a particular prediction.

Circuits in Language Models

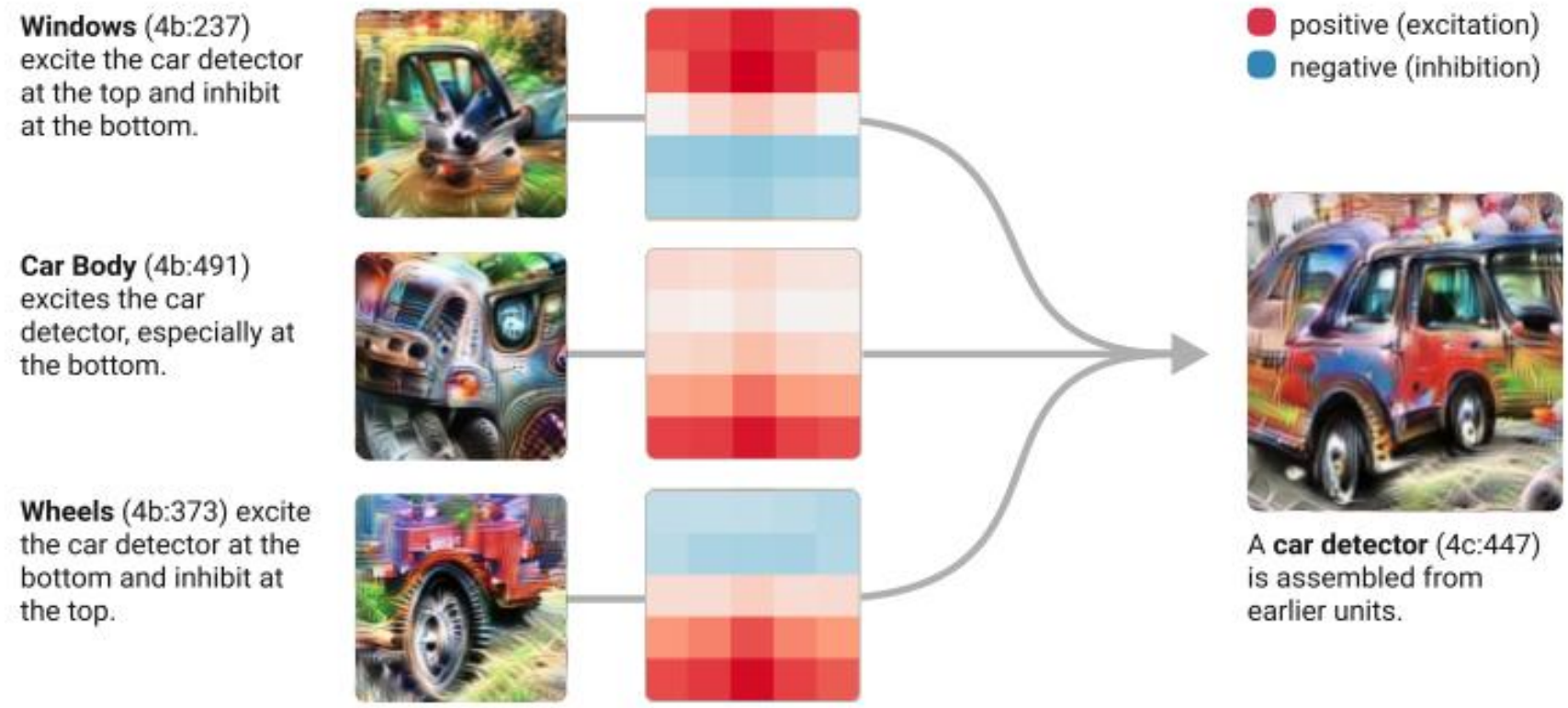
‘Circuits’

- A circuit is a *computational subgraph of a neural network*, with neurons (or, their linear combination) as nodes connected by the weighted edges that go between them in the original network.



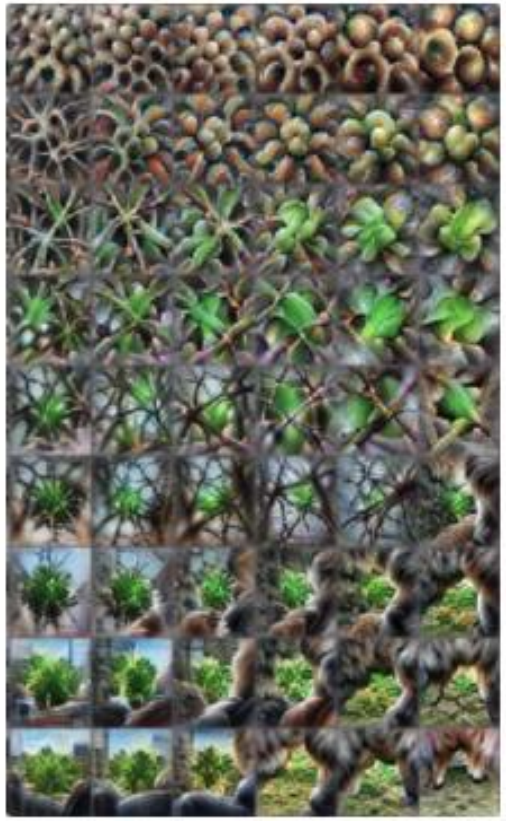
Olah, et al., "Zoom In: An Introduction to Circuits", Distill, 2020.

‘Circuits’



Olah, et al., "Zoom In: An Introduction to Circuits", Distill, 2020.

‘Circuits’



THREE SPECULATIVE CLAIMS ABOUT NEURAL NETWORKS

Claim 1: Features

Features are the fundamental unit of neural networks.
They correspond to directions. [1] These features can be rigorously studied and understood.

Claim 2: Circuits

Features are connected by weights, forming circuits. [2]
These circuits can also be rigorously studied and understood.

Claim 3: Universality

Analogous features and circuits form across models and tasks.

Left: An activation atlas [13] visualizing part of the space neural network features can represent.

Olah, et al., "Zoom In: An Introduction to Circuits", Distill, 2020.

Circuit in GPT-2 for IOI Task

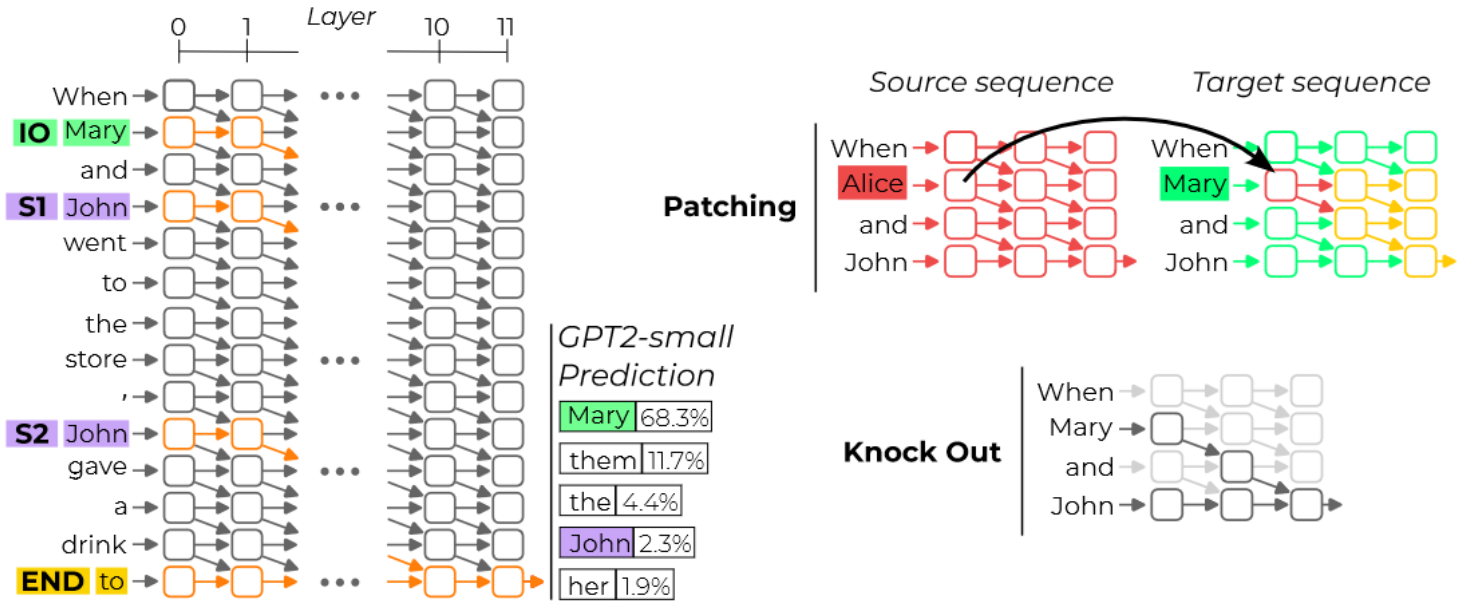


Figure 1: Left: We isolated a *circuit* (in orange) responsible for the flow of information connecting the indirect object 'Mary' to the next token prediction. The nodes are attention blocks and the edges represent the interactions between attention heads. Right: We discovered and validated this circuit using activation experiments, including both patches and knockouts of attention heads.

Wang, et al., Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 Small

Workflow for Finding Circuits

- Observe a behavior (or task) that a neural network displays, create a dataset that reproduces the behavior in question, and choose a metric to measure the extent to which the model performs the task.
- Define the scope of the interpretation, i.e. decide to what level of granularity(e.g. attention heads and MLP layers, individual neurons, whether these are split by token position) at which one wants to analyze the network. This results in a computational graph of interconnected model units.
- Perform an extensive and iterative series of patching experiments with the goal of removing as many unnecessary components and connections from the model as possible.

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits

- Observe a behavior (or task) that a neural network displays, create a dataset that reproduces the behavior in question, and choose a metric to measure the extent to which the model performs the task.
- Define the scope of the interpretation, i.e. decide to what level of granularity(e.g. attention heads and MLP layers, individual neurons, whether these are split by token position) at which one wants to analyze the network. This results in a computational graph of interconnected model units.
- Perform an extensive and iterative series of patching experiments with the goal of removing as many unnecessary components and connections from the model as possible.

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits

Task	Example Prompt	Output	Metric
1: IOI (Appendix F.2)	"When John and Mary went to the store, Mary gave a bottle of milk to"	"_John"	Logit difference

2: Docstring (Appendix H.1)	`def f(self, files, obj, state, size, shape, option):`
	`  """document string example`
	
	`  :param state: performance analysis`
	`  :param size: pattern design`
	`  :param`
		"_shape"	Logit difference

3: Greater-Than (Appendix G)	"The war lasted from 1517 to 15"	"18" or "19" or ... or "99"	Probability difference
4: tracr-xproportion (Appendix I.1)	`["a", "x", "b", "x"]`	`[0, 0.5, 0.33, 0.5]`	Mean Squared Error
5: tracr-reverse (Appendix I.2)	`[0, 3, 2, 1]`	`[1, 2, 3, 0]`	Mean Squared Error
6: Induction (Section 4.2)	"Vernon *Durs*ley and Petunia *Durs*"	"ley"	Negative log-probability

Table 1: Five behaviors for which we have an end-to-end circuit from previous mechanistic interpretability work, plus Induction. We automatically rediscover the circuits for behaviors 1-5 in Section 4. Tokens beginning with space have a "_" prepended for clarity.

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits

- Observe a behavior (or task) that a neural network displays, create a dataset that reproduces the behavior in question, and choose a metric to measure the extent to which the model performs the task.
- Define the scope of the interpretation, i.e. decide to what level of granularity(e.g. attention heads and MLP layers, individual neurons, whether these are split by token position) at which one wants to analyze the network. This results in a computational graph of interconnected model units.
- Perform an extensive and iterative series of patching experiments with the goal of removing as many unnecessary components and connections from the model as possible.

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits: Step 2 Examples

- To find circuits for the behavior of interest, one must represent the internals of the model as a **computational directed acyclic graph** (DAG).
- Current work chooses the abstraction level of the computational graph depending on the le vel of detail of their explanations of model behavior.
  - For example, at a coarse level, computational graphs can represent interactions between attention heads and MLPs.
  - At a more granular level they could include separate query, key and value activations, the interactions
  - between individual neurons, or have a node for each token position.

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits

- Observe a behavior (or task) that a neural network displays, create a dataset that reproduces the behavior in question, and choose a metric to measure the extent to which the model performs the task.
- Define the scope of the interpretation, i.e. decide to what level of granularity(e.g. attention heads and MLP layers, individual neurons, whether these are split by token position) at which one wants to analyze the network. This results in a computational graph of interconnected model units.
- Perform an extensive and iterative series of patching experiments with the goal of removing as many unnecessary components and connections from the model as possible.

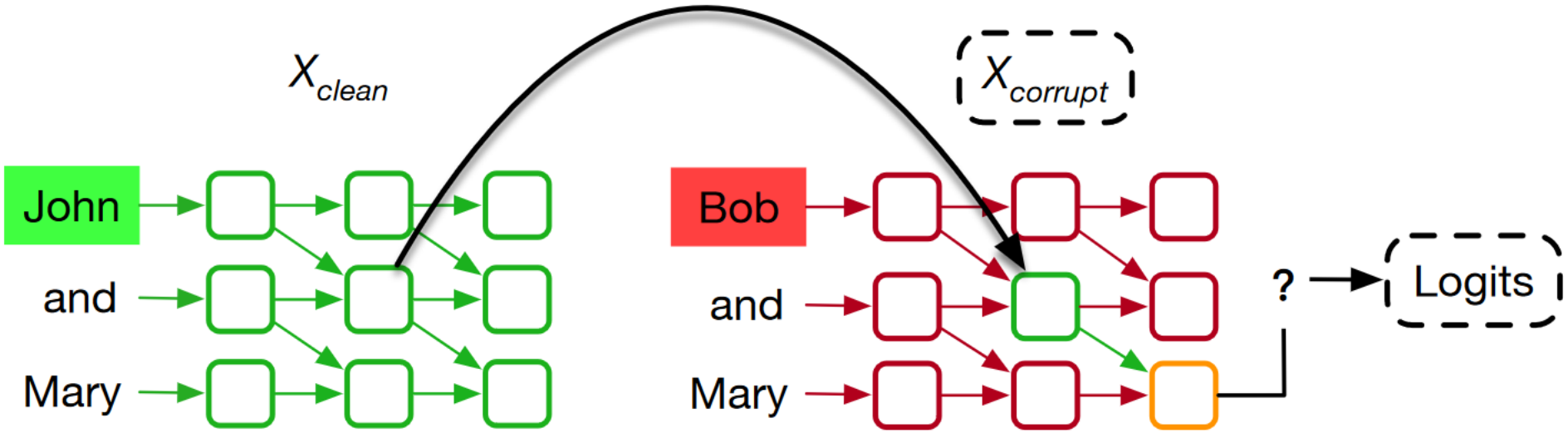
Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Workflow for Finding Circuits Step 3: Activation Patching

- The importance of nodes/edges are tested by using **recursive activation patching**:
  - overwrite the activation value of a node or edge with a corrupted activation,
  - run a forward pass through the model, and
  - compare the output values of the new model with the original model, using the chosen metric

Conmy et al., Towards Automated Circuit Discovery for Mechanistic Interpretability

Activation Patching



Zhang and Nanda., Towards Best Practices of Activation Patching in Language Models: Metrics and Methods

Activation Patching

The method involves a clean prompt (X_{clean}, e.g.,"The Eiffel Tower is in") with an associated answer r ("Paris"), a corrupted prompt ($X_{corrupt}$, e.g., "The Colosseum is in"), and three model runs:

1. **Clean run:** run the model on X_{clean} and cache activations of a set of given model components, such as MLP or attention heads outputs.
2. **Corrupted run:** run the model on $X_{corrupt}$ and record the model outputs.
3. **Patched run:** run the model on $X_{corrupt}$ with a specific model component's activation restored from the cached value of the clean run.

Finally, we evaluate the patching effect, such as P("Paris") in the patched run (3) compared to the corrupted run (2). Intuitively, corruption hurts model performance while patching restores it.

Patching effect measures how much the patching intervention restores performance, which indicates the importance of the activation.

Zhang and Nanda., Towards Best Practices of Activation Patching in Language Models: Metrics and Methods

Activation Patching: Metrics

- The **patching effect** is defined as the gap of the model performance between the corrupted and patched run, under an evaluation metric. Let *cl*, $*$, *pt* be the clean, corrupted and patched run.

  - Probability: $\mathbb{P}(r)$; e.g., $\mathbb{P}(\text{“Paris”})$. The patching effect is $\mathbb{P}_{\text{pt}}(r) - \mathbb{P}_{*}(r)$;
  - Logit difference: $\text{LD}(r, r') = \text{Logit}(r) - \text{Logit}(r')$; e.g., $\text{Logit}(\text{“Paris”}) - \text{Logit}(\text{“Rome”})$. The patching effect is given by $\text{LD}_{\text{pt}}(r, r') - \text{LD}_{*}(r, r')$. Following Wang et al. (2023), we always normalize this by $\text{LD}_{\text{cl}}(r, r') - \text{LD}_{*}(r, r')$, so it typically lies in $[0, 1]$, where 1 corresponds to fully restored performance and 0 to the corrupted run performance.
  - KL divergence: $D_{\text{KL}}(P_{\text{cl}}||P)$, the Kullback-Leibler (KL) divergence from the probability distribution of model outputs in the clean run. The patching effect is $D_{\text{KL}}(P_{\text{cl}}||P_{*}) - D_{\text{KL}}(P_{\text{cl}}||P_{\text{pt}})$.

Zhang and Nanda., Towards Best Practices of Activation Patching in Language Models: Metrics and Methods

Evaluating the Circuit: Faithfulness

- **Notation & Setup:**
  - M: The full, unablated neural network.
  - C: The hypothesized circuit (a specific sub-graph of *M*).
  - $X \sim p_{task}$: Input prompts drawn from the target task distribution.
  - $f(Output; X)$: The performance metric (e.g., logit difference).

- **Expected Performance:**
  - Full Model: *F(M)* = $\mathbb{E}_X[f(M(X); X)]$
  - Circuit Only: *F(C)* = $\mathbb{E}_X[f(C(X); X)]$

- **The Faithfulness Criterion:**

$$|F(M) - F(C)| \approx 0$$

  - *Interpretation:* A small difference proves the circuit (*C*) successfully captures the primary mechanisms the full model (*M*) uses to solve the task.

Wang et al, Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

Evaluating the Circuit: Completeness

- **The Problem with Faithfulness:** Redundant sub-circuits (e.g., C_1 OR C_2) can yield high faithfulness even if only one is found, leaving our understanding incomplete.

- **Notation Additions:**
  - *K*: A subset of circuit nodes to ablate ($K \subseteq C$).
  - *M* \ *K*: The full model with nodes *K* removed.
  - *C* \ *K*: The isolated circuit with nodes *K* removed.

- **The Completeness Criterion:**

$$|F(M \setminus K) - F(C \setminus K)| \approx 0 \quad \text{for all } K \subseteq C$$

- **Interpretation:**
  - *C* and *M* must degrade equally under identical knockouts.
  - **The Test:** If removing *K* destroys the circuit's performance but the full model survives, the model is using hidden backup mechanisms outside of *C*.

Wang et al, Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

Evaluating the Circuit: Minimality

- **The Limitation of Completeness:** A circuit is trivially faithful and complete if we simply define it as the entire model ($C = M$), which defeats the purpose of interpretability.

- **Notation Additions:**
  - v: A single candidate node currently included in the circuit ($v \in C$).
  - K: A subset of the remaining circuit nodes to ablate ($K \subseteq C \setminus \{v\}$).

- **The Minimality Criterion:**

$$\max_K \left| F(C \setminus (K \cup \{v\})) - F(C \setminus K) \right| \gg 0 \quad \text{for all } v \in C$$

- **Interpretation:**
  - Removing node *v* **must** cause a measurable drop in performance under at least one ablation condition (*K*).
  - **The Test:** If the performance drop is always near zero, node *v* is doing nothing useful and is a "free-rider." It fails minimality and must be pruned from *C*.

Wang et al, Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

The Evaluation Triad in Practice (Part 1)

- **The Setup:** Model $M = \{A, B, C\}$.
  - **Node A:** Primary mechanism (solves the task perfectly).
  - **Node B:** Backup mechanism (activates *only* if A is deleted).
  - **Node C:** Noise mechanism (irrelevant to the task).

- **Test 1: Faithfulness (Is it sufficient?)**
  - *Hypothesis:* Let's propose circuit $C_1 = \{A\}$.
  - *Calculation:* $F(\{A\}) \approx F(\{A, B, C\})$.
  - *Verdict:* **Passes.** Node A alone achieves full performance.

- **Test 2: Completeness (Is it exhaustive?)**
  - *The Knockout Test:* Ablate Node A $(K = \{A\})$.
  - *Calculation:* $F(M \setminus \{A\}) = F(\{B, C\})$, which remains high because Backup B takes over. But $F(C_1 \setminus \{A\}) = F(\emptyset)$, which drops to zero.
  - *Verdict:* **Fails.** C_1 is missing the backup mechanism. We must update our hypothesis to $C_2 = \{A, B\}$.

Wang et al, Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

The Evaluation Triad in Practice (Part 2)

- **Recap:** Our current hypothesis is $C_2 = \{A, B\}$. It is both faithful and complete.

- **Test 3: Minimality (Is it strict?)**
  - *The Bloat Test:* Suppose another researcher proposes $C_3 = \{A, B, C\}$. It is also faithful and complete. How do we disprove it? Let's test Node C.
  - *Calculation:* For any ablation set K, removing C changes absolutely nothing. For example: $|F(\{A, B\}) - F(\{A, B, C\})| = 0$.
  - *Verdict:* **Fails.** Node C causes no measurable performance drop when removed. It is a "free-rider" doing no useful work.

- **The Conclusion:**
  - Node C must be pruned.
  - The final, perfect circuit that passes all three criteria is $C = \{A, B\}$.

Wang et al, Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

Automatic Circuit DisCovery (ACDC)

Algorithm 1: The ACDC algorithm.

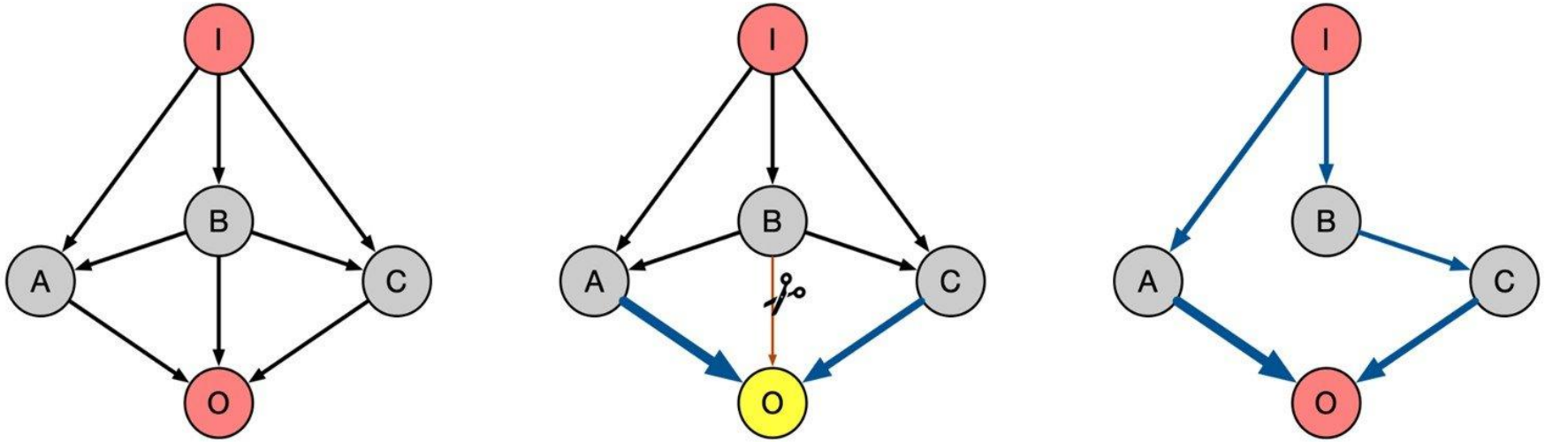
Data: Computational graph G, dataset $(x_i)_{i=1}^n$, corrupted datapoints $(x_i')_{i=1}^n$ and threshold $\tau > 0$.

Result: Subgraph $H \subseteq G$.

```
H ← G                                   // Initialize H to the full computational graph
H ← H.reverse_topological_sort()        // Sort H so output first
for v ∈ H do
    for w parent of v do
        H_new ← H \ {w → v}             // Temporarily remove candidate edge
        if D_KL(G||H_new) − D_KL(G||H) < τ then
            H ← H_new                   // Edge is unimportant, remove permanently
        end
    end
end
return H
```

Commy et al, Towards Automated Circuit Discovery for Mechanistic Interpretability

Automatic Circuit DisCovery (ACDC)



(a) Choose computational graph, task, and threshold τ.

(b) At each head, prune unimportant connections.

(c) Recurse until the full circuit is recovered.

Commy et al, Towards Automated Circuit Discovery for Mechanistic Interpretability

Automatic Circuit DisCovery (ACDC)

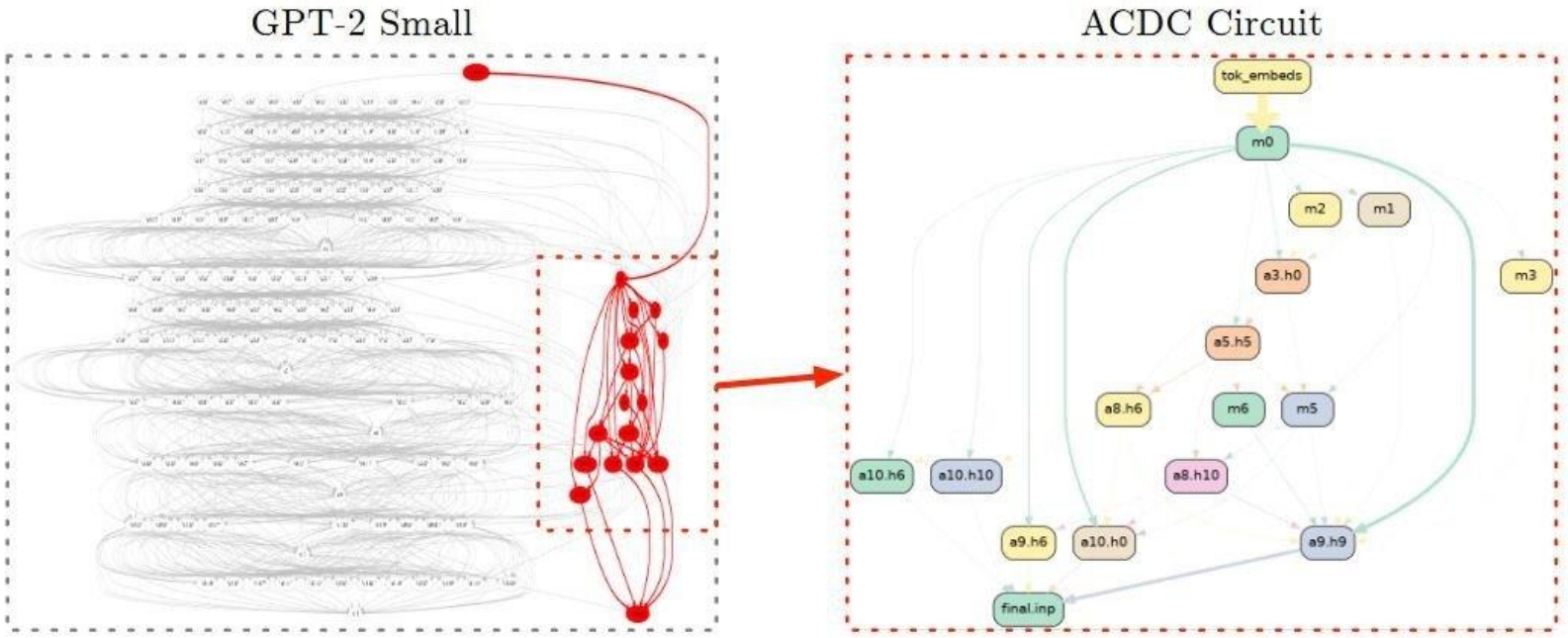


Figure 1: **Automatically discovering circuits with ACDC.** *Left:* a computational graph for GPT-2 Small, with a recovered circuit for the IOI task highlighted in red. Only edges between adjacent layers are shown. *Right:* the recovered circuit with labelled nodes. All heads recovered were identified as part of the IOI circuit by Wang et al. (2023). Edge thickness is proportional to importance.

Commy et al, Towards Automated Circuit Discovery for Mechanistic Interpretability

Neuron-Based Interpretability

Decoding the *Black Box* at the Microscopic Level

Neuron-Based Interpretability: Motivation

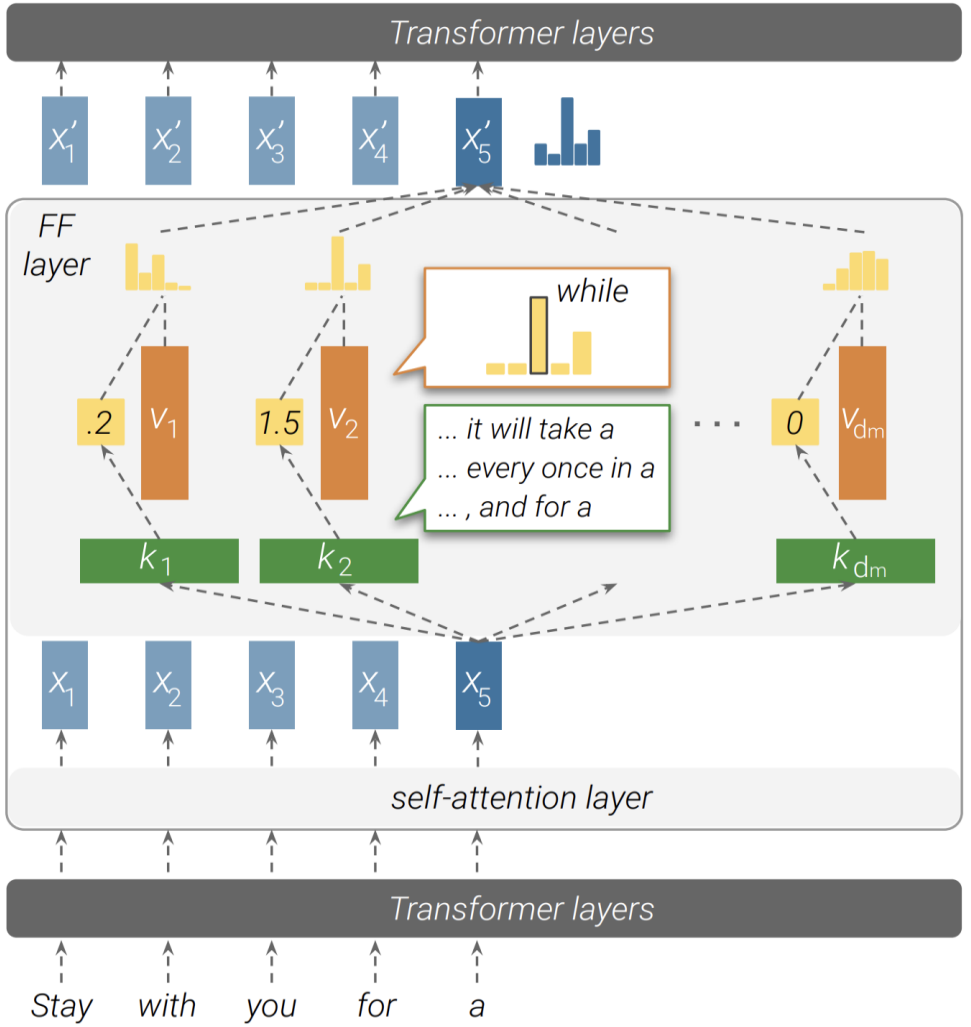
- **The Focus Shift:**
  - Previous methods explain how Attention Heads *route* information.
  - The open problem is determining where actual factual knowledge is *stored*.

- **The Parameter Allocation:**
  - Feed-Forward Networks (MLPs) account for roughly two-thirds of a standard Transformer's total parameters.
  - Historically, these layers have been treated as uninterpretable black boxes.

- **The Research Question:**
  - Do individual MLP neurons map to distinct, human-understandable concepts?

Feed-Forward Layers as Key-Value Memories

- **The Causal Role of the MLP**
  - **Hypothesis:** Feed-Forward Networks act as explicit, readable **Key-Value memory banks**.
- **Keys (The "If" Condition)**
  - Act as **Pattern Detectors**.
- **Values (The "Then" Action)**
  - Act as **Output Predictors**.



Geva et. al, Transformer Feed-Forward Layers Are Key-Value Memories *EMNLP 2021*

MLP as Key-Value Memories

- **The Key-Value Operation**

$$\mathrm{FF}(x) = f(x \cdot K^\top) \cdot V$$

- **Variable Deconstruction**
  - ***K* (Keys):** Pattern detectors
  - ***f* (Activation):** Sparsity gate (e.g., ReLU)
  - ***V* (Values):** Output predictors

- **Layer Composition**

$$y^l = \sum_i \mathrm{ReLU}(x^l \cdot k_i^l) \cdot v_i^l + b^l$$

Geva et. al, Transformer Feed-Forward Layers Are Key-Value Memories *EMNLP 2021*

Empirical Evidence: What Do Keys Actually Detect?

The Depth Progression

- **Lower Layers (Shallow):** Simple surface-level patterns and n-grams.
  - *Example:* Firing only when a sentence ends in "substitutes".
- **Middle Layers (Hybrid):** Combinations of specific vocabulary and context.
  - *Example:* Firing for military contexts ending in the word "base".
- **Upper Layers (Semantic):** Abstract, conceptual topics regardless of exact vocabulary.
  - *Example:* Firing for text discussing TV shows or time ranges.

Key	Pattern	Example trigger prefixes
$\mathbf{k}^1_{449}$	Ends with *"substitutes"* (shallow)	*At the meeting, Elton said that "for artistic reasons there could be no substitutes* *In German service, they were used as substitutes* *Two weeks later, he came off the substitutes*

Geva et. al, Transformer Feed-Forward Layers Are Key-Value Memories *EMNLP 2021*

Values as Output Predictors

- **The Core Concept**
  - Keys = Pattern Detectors.
  - **Values = Stored Responses**.

- **Vocabulary Projection**
  - Each value vector contains an uncalibrated probability distribution over the entire output vocabulary.
  - We reveal this distribution by multiplying it by the output embedding matrix E:

$$p_i^l = \text{softmax}(v_i^l E)$$

- **Example**

Value	Prediction	Precision@50	Trigger example
$\mathbf{v}_{222}^{15}$	*each*	68%	*But when bees and wasps resemble each*

- **Takeaway:** FFNs are not just abstract transformers; they explicitly store direct next-token predictions mapped to specific contexts.

Geva et. al, Transformer Feed-Forward Layers Are Key-Value Memories, *EMNLP 2021*

Issues

- When MLPs or neurons are considered as units of interpretability, *polysemanticity of neurons* becomes an issue.
  - A single neuron may be responsible for encoding multiple concepts or functions.
  - How do we then disentangle the components of a model responsible for a particular behaviour?
  - This is also crucial if we want to control only a particular behaviour of the model without affecting its other capabilities.